
∞ REASONING-TIME TRAINING: LEARNING BEFORE A SINGLE PROBLEM ENDS

MMLA FOCUS PAPER

Junyi Zou^{1,*} • Avrova Donz^{1,2,*}

¹MMLA-org ²Communication University of China (CUC), No. 1 Dingfuzhuang East Street, Chaoyang District, Beijing 100024, China

*Equal contribution

Email: zoujunyi@zjydiary.cn · avrovaDonz@icloud.com

Project: github.com/MMLA-org/mmla-memory

ABSTRACT

Can a reasoning system learn from one failed attempt early enough to change how it attacks the same unresolved problem? More sampling, a larger search tree, a longer prompt, or an edited memory can improve a later attempt without changing the policy that generated it. We therefore define *strict Reasoning-Time Training* (RTT) through an auditable temporal witness: feedback from an attempt updates a declared policy state, the update completes before the episode ends, and a later attempt actually reads the post-update state. A typed stochastic process makes policy, context, memory, workspace, verifier, lifetime, and budget explicit. Under stated assumptions, we prove separations from fixed-policy search and memory-only mutation; show why finite transcripts do not identify the mechanism; derive randomized intervention estimators, reset and no-read consequences, verifier-error bounds, a score-function identity, group-relative degeneracy, and a cumulative drift bound; and state rollback and falsification obligations. These results define what would count as within-problem learning. They do not report an RTT experiment or establish utility over search, prompting, memory, or offline training.

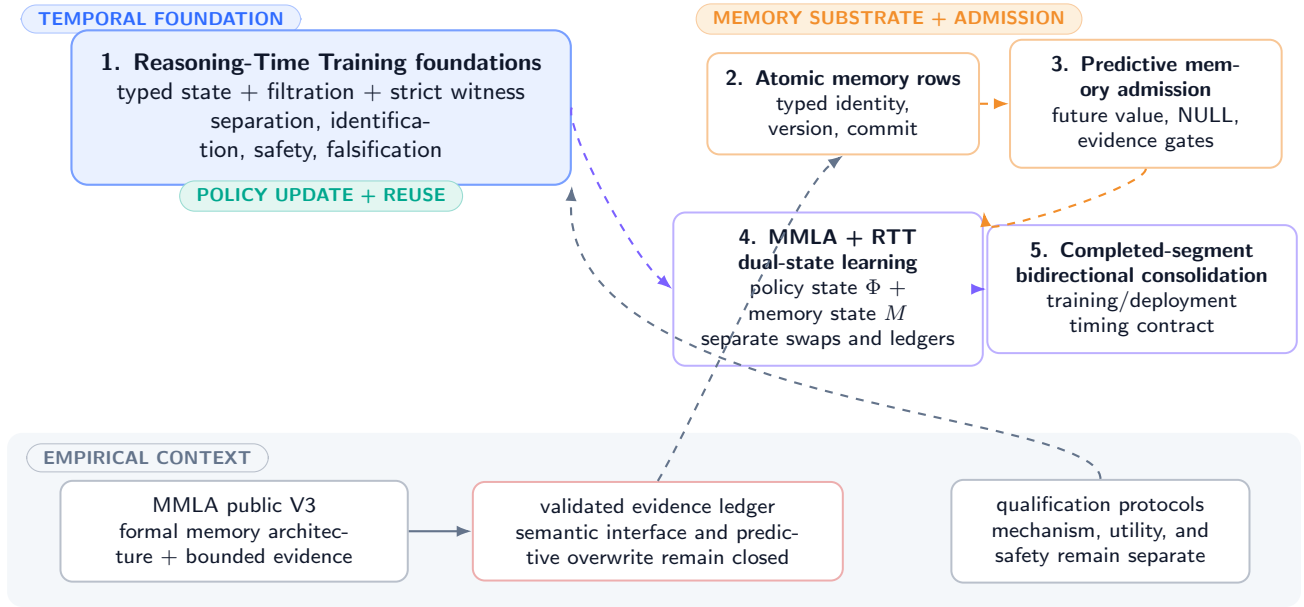
1 Introduction

Suppose a system receives one difficult problem, produces an unsuccessful attempt, observes a compiler error or verifier score, and later succeeds. The success alone does not tell us what changed. The system may have sampled a luckier trajectory from the same policy, expanded a larger search tree, appended the error to its prompt, stored a reflection in external memory, or changed a state that parameterizes the policy. All five mechanisms can be useful. Only the last is a candidate for the phenomenon studied here, and even then the update must occur and be reused before that same problem is resolved.

We use *Reasoning-Time Training* (RTT) for this within-problem temporal regime. The name is descriptive rather than a claim of lexical priority. “Training” refers to a change in a declared policy-generating state, not necessarily to backpropagation. “Reasoning time” is the interval between admission of one problem instance and its resolution or abandonment. The boundary is therefore operational: state type, event order, read trace, reset scope, and intervention response must be inspectable.

MMLA / RTT focus-paper family: conceptual dependencies

five complementary formal lenses; arrows organize concepts and do not transfer empirical validation



Formal dependencies organize the family; each implementation claim still requires its own intervention and evidence.

Figure 1: **Five-paper conceptual map.** This paper supplies the temporal and causal foundations used by the four companion focus papers. Arrows encode mathematical dependencies, not empirical validation.

The relationship to MMLA is deliberately narrow. MMLA formalizes a bounded authoritative memory between context and slow weight updates [12]. The public evidence record does not qualify same-checkpoint semantic memory or learned predictive overwrite. We do not import those component results as evidence for RTT. Instead, we separate policy mutation from external-memory mutation so that a combined system can ablate their causal roles independently.

1.1 Contributions

The contributions are:

1. a typed stochastic process for a single unresolved problem, with attempt, feedback, policy, context, memory, workspace, verifier, lifetime, and budget objects;
2. a strict witness definition that requires an in-lifetime policy-state update and an actually post-update-policy-conditioned later attempt;
3. separation and non-identifiability results clarifying what transcripts alone cannot establish;
4. causal qualification designs and estimands that isolate update use, feedback semantics, reset leakage, verifier reliability, and local harm;
5. an update-operator taxonomy, outer objective, budget ledger, rollback contract, implementation obligations, and a staged falsification ladder.

Formal statements carry their assumptions locally. Status tags distinguish definitions, derived results, design prescriptions, and future tests; none converts a theorem into implementation evidence. No experiment in this document demonstrates RTT, no theorem guarantees that an optimizer finds a useful update, and no result establishes superiority over search, context refinement, memory, or offline training. Policy mutation may be unnecessary, unstable, or inefficient. We neither relabel arbitrary cache mutation as training nor infer strict RTT from a method name.

1.2 Episode boundary and paper map

The unit is a *problem episode*: one admitted instance, its ordered attempts, its within-episode feedback, and a terminal resolution or abandonment decision. A benchmark dataset is a collection of such episodes, not one giant problem. An update learned from earlier dataset items and used on later items is test-time or continual adaptation, but is not strict RTT for a later item unless that later item itself contains an update-reuse witness before its own endpoint. Conversely, a harmful within-problem update can satisfy the mechanism definition even though it fails every utility gate. Mechanism qualification

and benefit qualification are separate questions throughout.

Section 2 specifies the typed process; Section 3 gives the strict witness; Sections 4–5 establish separations and identification limits; and the remaining sections develop update objectives, safety contracts, implementation obligations, related work, and falsification tests. Full proofs, symbols, and audit-oriented derivations appear in the appendices.

2 A Typed Within-Problem Stochastic Process

2.1 Probability space and typed interfaces

Fix a probability space $(\Omega, \mathcal{A}, \mathbb{P})$. A problem episode draws $X : \Omega \rightarrow \mathcal{X}$ and is assigned an immutable episode identifier $I \in \mathcal{I}$. The identifier distinguishes a retry of the same problem from a later, merely similar item. Within the episode, attempts are indexed by $j \in \mathbb{N}$ and token/tool microsteps by $t \in \mathbb{N}$. The basic spaces are collected in Table 1; they are measurable spaces even when implemented as structured byte records.

Object	Space	Intended content	Required boundary
Problem X	$\mathcal{X}$	Immutable task statement, tools, and success contract.	Same episode identifier across witness attempts.
Observation $O_{j,t}$	$\mathcal{O}$	Tokens, tool returns, environment state visible by time (j, t) .	Timestamped; no future returns.
Action $A_{j,t}$	$\mathcal{A}$	Token, tool call, branch, halt, or answer action.	Sampled from the declared policy kernel.
Attempt Z_j	$\mathcal{Z}$	Finite marked trajectory of observations and actions.	Start/end and policy-state receipt recorded.
Feedback Y_j	$\mathcal{Y}$	Verifier output, execution result, critique, or reward record.	Provenance and availability time recorded.
Policy state Φ_j	$\mathcal{P}$	Mutable state directly parameterizing the action kernel.	Versioned, bounded, intervenable, and read-traced.
Context C_j	$\mathcal{C}$	Prompt and transient in-context transcript.	Not silently counted as policy state.
Memory M_j	$\mathcal{M}$	External or resident episodic/semantic records.	Typed separately from policy state.
Workspace W_j	$\mathcal{W}$	Files, search tree, scratchpad, caches, tool state.	Charged and reset-scoped explicitly.
Budget B_j	$\mathcal{B} \subseteq \mathbb{R}_+^d$	Remaining tokens, calls, time, energy proxy, update steps.	Monotone ledger with hard stop.

Table 1: **Typed state decomposition** (DEFINITION). Physical storage does not determine the type. The causal interface and declared read path do.

Let the full state immediately before attempt j be

$$S_j = (X, I, C_j, M_j, W_j, \Phi_j, \Xi_j, B_j), \quad (1)$$

where Ξ_j contains random-generator state, optimizer moments, routing metadata, and other auxiliary variables. Excluding Ξ_j from an audit does not make it causally irrelevant. Every persistent field that could carry information between experimental arms belongs either in the declared state or in a leakage inventory.

Definition 2.1 (Policy kernel and functional policy state). **DEFINITION** For each microstep, the system exposes a stochastic kernel

$$\pi_\phi(da \mid x, h, c, m, w, b), \quad (2)$$

where h is the causally available within-attempt history. A declared component Φ is a *functional policy state* on a history set $\mathcal{H}_0$ if there exist $h \in \mathcal{H}_0$, admissible states ϕ, ϕ' , and fixed (x, c, m, w, b) such that

$$\text{TV}(\pi_\phi(\cdot \mid x, h, c, m, w, b), \pi_{\phi'}(\cdot \mid x, h, c, m, w, b)) > 0. \quad (3)$$

The declaration must identify the serialization, version, update rule, and action-kernel call site. A semantic relabeling of context or memory without such an interface is insufficient.

This definition is intentionally substrate-neutral. Full weights, a low-rank adapter, a learned-optimizer state, a recurrent fast-weight state, or a hypernetwork code can all be functional policy state. Conversely, a vector called an “adapter” that is never read is not functional on the observed episode. External memory can influence actions after retrieval, but its mutation remains a memory operation unless it is also the declared direct policy-generating state under an auditable interface.

2.2 Filtration, attempt times, and resolution

Let $\{\mathcal{F}_n\}_{n \geq 0}$ be a discrete global filtration at event-clock resolution. It contains the problem and every observation, action, random seed draw, verifier return, state version, and external side effect available by event n . For attempt j , let σ_j and ϵ_j be start and end stopping times with $\sigma_j \leq \epsilon_j < \sigma_{j+1}$ on episodes that retry. Let u_j be the completion time of the update triggered after attempt j . The resolution time τ is a stopping time taking values in $\mathbb{N} \cup \{\infty\}$; resolution includes verified success, explicit abandonment, budget exhaustion, or a safety stop.

Within an attempt, $\mathcal{F}_{j,t} := \mathcal{F}_{n(j,t)}$ denotes the information set before action $A_{j,t}$. Causality requires

$$A_{j,t} \text{ is sampled from an } \mathcal{F}_{j,t} \text{-measurable kernel,} \quad O_{j,t+1} \in \mathcal{F}_{j,t+1}. \quad (4)$$

Feedback Y_j becomes available at a stopping time $v_j \geq \epsilon_j$ and is $\mathcal{F}_{v_j}$ -measurable. An admissible update has the form

$$(\Phi_{j+1}, \Xi_{j+1}, \rho_j) = U_j(\Phi_j, \Xi_j, X, Z_{\leq j}, Y_{\leq j}, B_j; R_j), \quad (5)$$

where R_j is fresh update randomness revealed no later than u_j , and ρ_j is a receipt containing input/output hashes, times, costs, and status. The entire right-hand side must be $\mathcal{F}_{u_j}$ -measurable. In particular, no observation after u_j may select or tune the update used before that observation.

Strict RTT causal timeline inside one unresolved problem

event order is part of the definition: feedback, update, and actual reuse all precede the registered resolution time

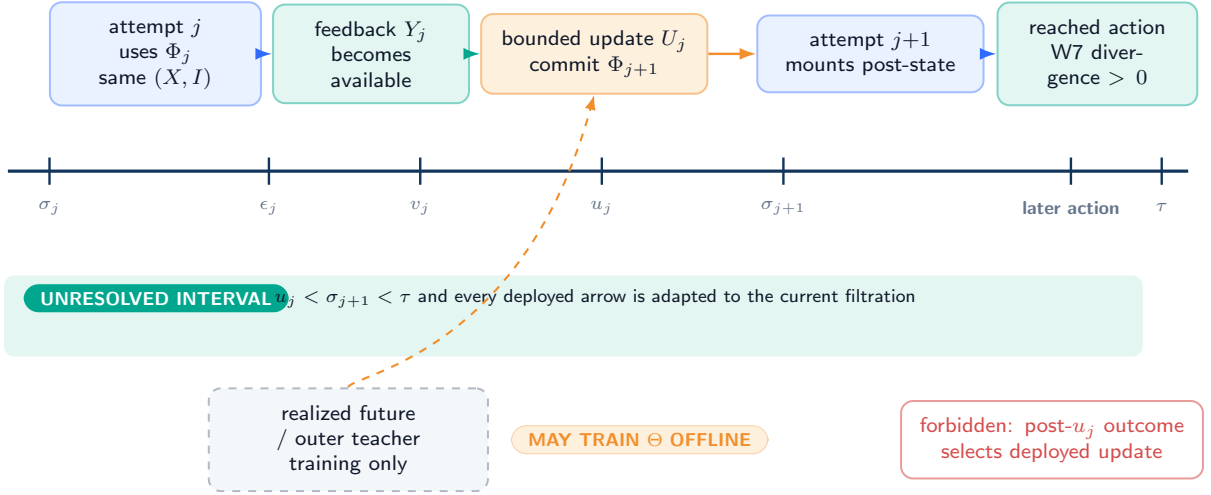


Figure 2: **Strict within-problem causal timing (DEFINITION)**. Solid arrows are deployed, causally available paths. Attempt j ends, feedback becomes available, and update U_j commits while the same episode is unresolved; only then may attempt $j+1$ invoke Φ_{j+1} . The dashed amber future branch is allowed only for an outer training teacher and may not enter the deployed update.

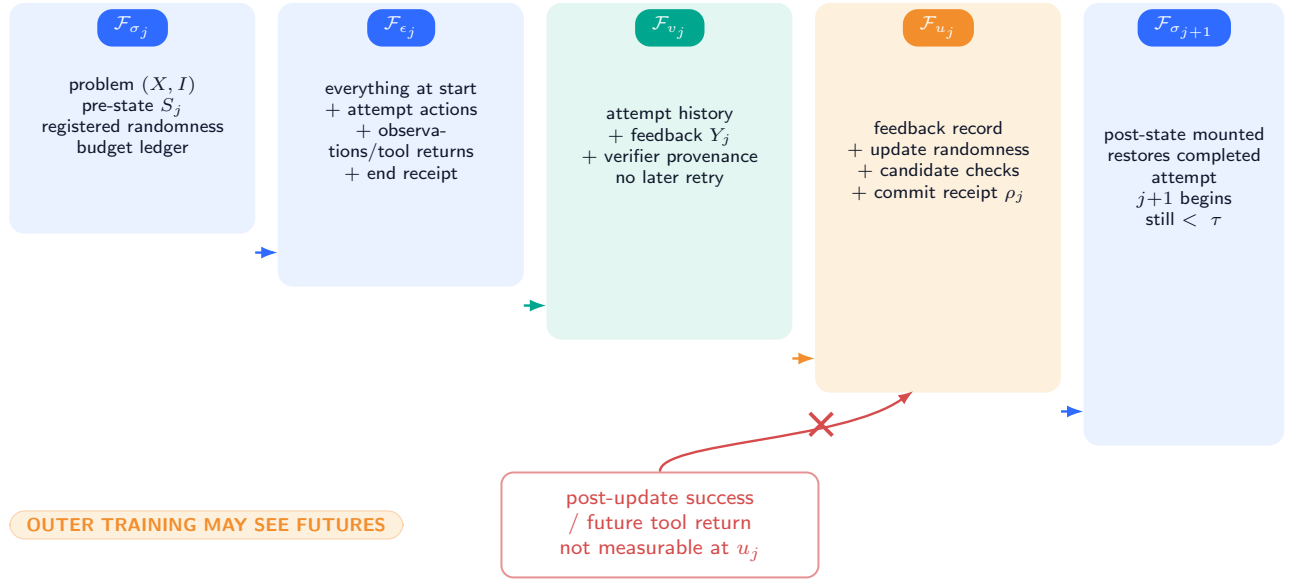
Definition 2.2 (Unresolved interval and lifetime). **DEFINITION** The episode is unresolved at event n on $\{n < \tau\}$. A policy-state version ϕ has a declared lifetime interval $L(\phi) = [\ell^-(\phi), \ell^+(\phi))$ in event time. A within-problem version must satisfy

$$\sigma_1 \leq \ell^-(\phi) < \ell^+(\phi) \leq \tau + 1, \quad (6)$$

unless an explicitly separate cross-problem continual-learning protocol owns the carryover. Retaining a version after τ silently changes the regime.

Within-problem filtration: what may influence each event

nested information sets grow with revealed observations; causality forbids arrows from unrevealed future events



Nested panels depict inclusion of information, not equal wall-clock duration. Distributed systems must record the causal partial order.

Figure 3: **Information sets and admissible arrows (DEFINITION)**. Each blue filtration panel contains only events already revealed. Policy actions, feedback processing, and the update receipt are adapted to the current filtration. The crossed coral arrow is future leakage: later success or a post-update tool result cannot be used to choose an earlier deployed update.

2.3 Standing assumptions for formal results

Results invoke only the assumptions they cite; no assumption is globally true by typography.

Assumption 2.3 (Typed observability). **PROVED UNDER STATED ASSUMPTIONS** The declared state components in Equation (1), their versions, and their read/write events are logged without omission for the analyzed episode.

Assumption 2.4 (Causal availability). **PROVED UNDER STATED ASSUMPTIONS** Actions and updates are adapted to $\{\mathcal{F}_n\}$, and the resolution time τ is a stopping time for that filtration.

Assumption 2.5 (Stable episode identity). **PROVED UNDER STATED ASSUMPTIONS** Witness attempts share the same immutable pair (X, I) ; a generated paraphrase or neighboring dataset item is not silently substituted.

Assumption 2.6 (Well-defined interventions). **PROVED UNDER STATED ASSUMPTIONS** When a result compares state versions or feedback arms, all non-target state fields and environment kernels are either held fixed, randomized as specified, or included in the estimand.

Assumption 2.7 (Positivity and consistency). **PROVED UNDER STATED ASSUMPTIONS** Every randomized experimental arm has positive assignment probability, and the observed outcome equals the potential outcome under the assigned arm without cross-episode interference.

Assumption 2.8 (Bounded evaluation). **PROVED UNDER STATED ASSUMPTIONS** Utility and harm variables used in stability bounds lie in $[0, 1]$, or are rescaled with an explicitly reported range.

3 Strict Reasoning-Time Training

3.1 A witness is a trace plus a functional state test

The definition below is intentionally stronger than “an optimizer ran.” A no-op optimizer, an update committed after resolution, or an updated adapter that is never invoked does not establish within-problem learning.

Definition 3.1 (Strict RTT witness). **DEFINITION** A strict RTT witness for attempt index j is a record

$$\mathfrak{W}_j = (I, j, \epsilon_j, v_j, u_j, \sigma_{j+1}, \tau, h_j^-, h_j^+, \rho_j, \mathcal{R}_{j+1}, d_j) \quad (7)$$

that satisfies all of the following:

- W1. Same problem.** Attempts j and $j+1$ carry the same immutable (X, I) .
- W2. Available feedback.** Attempt j ends at ϵ_j , feedback Y_j is available by v_j , and $Y_j \in \mathcal{F}_{v_j}$.
- W3. Nontrivial policy-state transition.** Update U_j commits at $u_j \geq v_j$, producing a version change $h_j^- := H(\Phi_j) \neq H(\Phi_{j+1}) =: h_j^+$ under a collision-resistant serialization hash and a valid receipt ρ_j .
- W4. Causal update.** $(\Phi_{j+1}, \Xi_{j+1}, \rho_j)$ is $\mathcal{F}_{u_j}$ -measurable; no event after u_j selects the committed update.
- W5. Problem still unresolved.** $u_j < \sigma_{j+1} < \tau$.
- W6. Actual post-update read.** The later attempt contains a signed read trace $\mathcal{R}_{j+1}$ showing that the action-kernel call loaded version h_j^+ , not merely that the version existed.
- W7. Functional reuse on a reached history.** For at least one reached pre-action history $H_{j+1,t}$, with (X, C, M, W, B) fixed to their recorded values,

$$d_j := \text{TV}\left(\pi_{\Phi_{j+1}}(\cdot \mid H_{j+1,t}), \pi_{\Phi_j}(\cdot \mid H_{j+1,t})\right) > 0. \quad (8)$$

The distance may be computed exactly, lower-bounded with a registered probe, or established by a randomized state-swap test whose uncertainty is reported.

- W8. Charged lifetime.** Update, retained state, retry, and verification costs are charged to B_j , and the state version expires or transfers under a declared rule at τ .

No positive reward, correctness, or improvement condition is part of the mechanism definition.

The hash inequality in W3 is necessary but not sufficient: different bytes can implement the same kernel. W6 catches dead updates; W7 catches semantically inert updates. Conversely, W7 does not show that the change helped. It only shows that a later attempt was genuinely conditioned on a policy state produced from earlier within-problem feedback.

Definition 3.2 (Syntactic, functional, and beneficial qualification). **DEFINITION** A trace satisfying W1–W6 and W8 is a *syntactic update–reuse trace*. It becomes a *strict functional RTT witness* only with W7. A *beneficial causal RTT effect* additionally requires a prespecified outcome estimand and an intervention or randomized design supporting a positive effect under its assumptions. These three levels must not be conflated.

Item	Required evidence	Typical falsifier	Status if absent
W1	Immutable problem/episode receipt	New item or regenerated task ID	Different regime
W2	Timestamped feedback provenance	Feedback created after update	Causal failure
W3	Input/output state hashes and commit	No-op or failed commit	Not strict RTT
W4	Filtration/future-read audit	Later success tunes earlier update	Leakage
W5	$u_j < \sigma_{j+1} < \tau$	Update after answer acceptance	Too late
W6	Kernel call reads post-state	Adapter never mounted	Dead update
W7	Exact/probed state-swap divergence	Zero divergence on reached histories	Functionally inert
W8	Cost/lifetime ledger	Uncharged retained state	Protocol invalid

Table 2: **Strict witness audit** (**DEFINITION**). Each row has an independently inspectable artifact and an explicit failure interpretation.

3.2 Fresh-proposal criterion

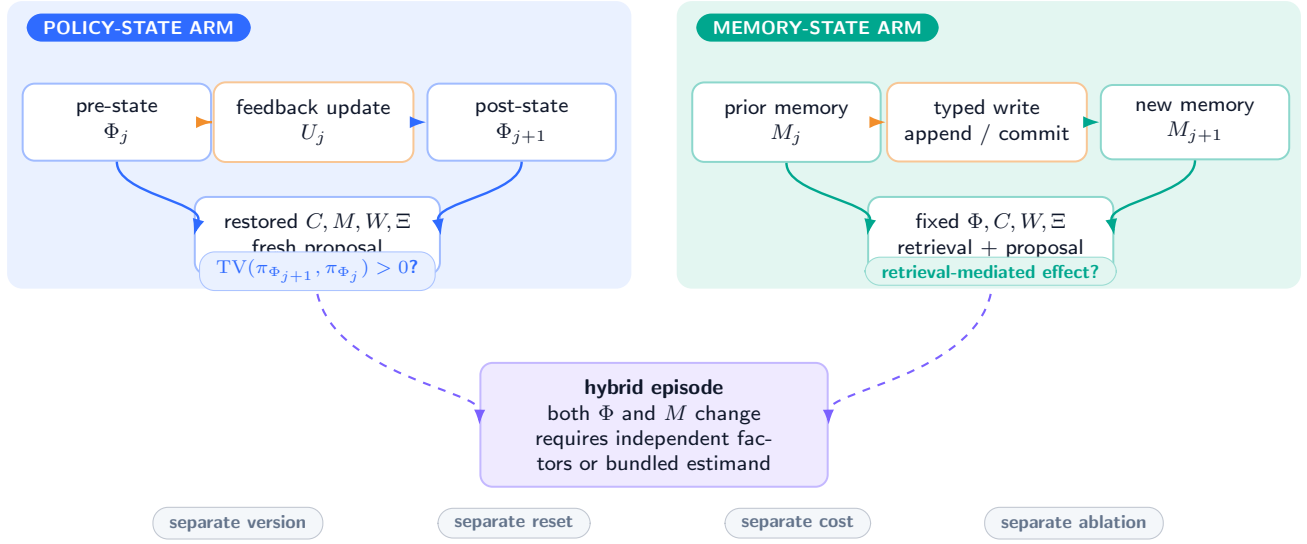
Search workspaces can strongly influence which node is expanded next. To avoid declaring every search statistic a learned policy, the state partition uses a fresh-proposal intervention.

Definition 3.3 (Fresh-proposal policy-state criterion). **DEFINITION** Let r restore (X, C, M, W, B) to a registered pre-attempt snapshot while independently choosing policy version ϕ . A mutable state component qualifies as policy state for the episode only if, under some admissible restored snapshot and common randomization scheme, swapping that component changes the distribution of a *fresh proposal* before any updated search node, appended reflection, or new memory record is read. Candidate-specific tree values, visited-node sets, and cached tool outputs are workspace state by default.

This criterion does not assert that workspace search is inferior. It enforces a stable meaning: RTT changes how new reasoning actions are proposed; search changes which already represented possibilities are explored or selected. A hybrid can do both and should report both.

Two mutable carriers, two causal questions

policy state changes the fresh proposal kernel; memory changes what a retrieval path can return



Physical co-location does not merge scientific roles; declared interfaces and controlled swaps determine the carrier.

Figure 4: **Mutable policy state versus external memory and workspace (DEFINITION).** The navy state-swap holds context, memory, workspace, and randomness fixed and asks whether a fresh proposal distribution changes. Teal memory swaps test retrieval-mediated effects separately. A hybrid path may be deployed, but each state type needs its own version, reset, cost, and ablation receipt.

3.3 A worked strict witness

Consider a coding instance with a deterministic compiler verifier. Attempt 1 produces program z_1 and error y_1 . A rank- r adapter $\Phi_1 = (A_1, B_1)$ is updated to Φ_2 by K bounded optimizer steps using only (X, z_1, y_1) , and the commit receipt is written before retry. The prompt, external memory, compiler cache, and workspace are then restored to their registered pre-attempt values. Attempt 2 mounts Φ_2 . A common-random-number logit probe at a reached prefix reports positive total variation between the distributions under Φ_2 and restored Φ_1 . This is a strict functional witness regardless of whether program 2 passes.

The example also shows why “the second answer differed” is inadequate. Different sampling randomness can change an answer with no update, and hidden compiler caches can change it despite a reset. The receipt must cover the state version, actual call path, controlled fields, and measurement uncertainty.

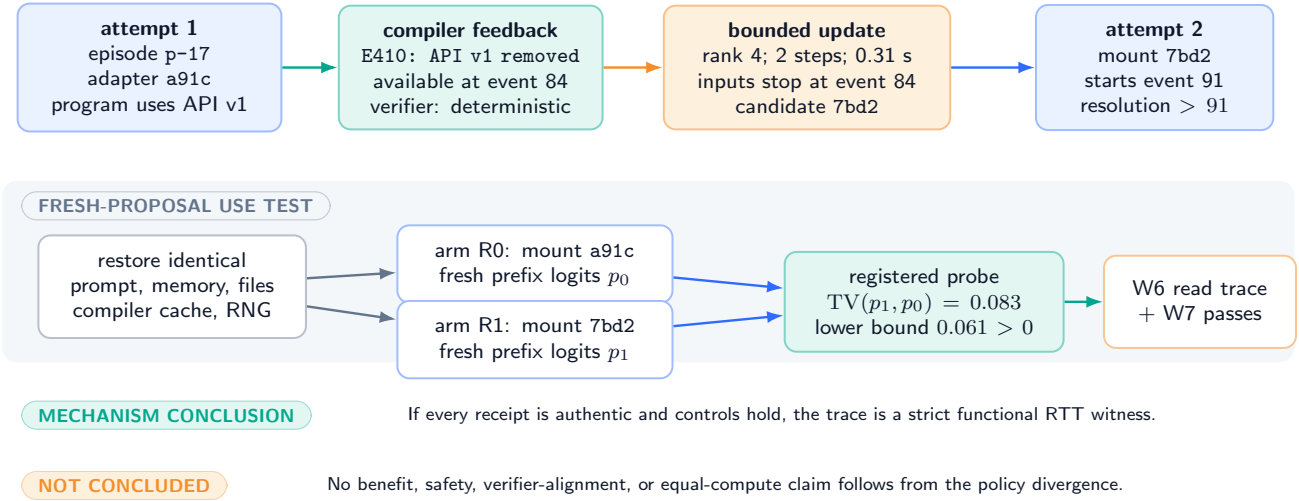
3.4 Hybrid systems and dual accounting

A system may update Φ , append a reflection to C , write M , and expand W after the same attempt. Such a trace is not rejected, but the total causal effect is then a mixture. The minimal factorial decomposition uses independent restore/swap factors for policy state and non-policy information carriers. If cost permits, context, memory, workspace, verifier feedback, and policy version receive separate randomized factors. If cost does not permit, the reported estimand must be the bundled effect; it may not be called a policy-update effect.

Worked strict witness: compiler feedback changes a bounded adapter

FORMALIZED EXAMPLE / NOT MEASURED

hash fragments, cost, and divergence values are illustrative only



A real bundle must include full hashes, signatures, uncertainty construction, arm assignments, and every charged resource.

Figure 5: **Worked multi-attempt trace (formalized example, not measured)**. Attempt 1 yields compiler feedback; a bounded adapter update commits while the episode is unresolved; attempt 2 actually mounts the new version. The lower restored-state branch is the required policy-use probe. All numerical hashes, costs, and distances are illustrative and carry no empirical claim.

4 Separation from Search, Context, and Memory

4.1 Mechanism taxonomy

The adjacent regimes differ along three axes: what state changes, when it changes, and which future unit uses it. Table 3 and Figure 6 use the strict witness rather than community labels.

Regime	Primary mutable carrier	Reuse horizon	Strict RTT condition
Fixed-policy sampling/search	W (samples, nodes, scores)	Same problem	No, unless Φ also changes and passes fresh-proposal reuse.
Prompt/context refinement	C	Same problem/session	No when Φ is fixed.
External memory / RTMA	M	Later attempts or problems	No when only M changes.
Classical TTT/adaptation	Φ	Current sample, batch, or stream	Only episodes with update and reuse before that same problem ends.
Continual/online learning	Φ	Later problems	Not strict RTT for the earlier completed problem; may contain nested RTT episodes.
Strict RTT	Φ	Later attempt on same unresolved (X, I)	Yes, if all W1–W8 hold.

Table 3: **Operational regime taxonomy (DEFINITION)**. Names used by individual papers do not override the trace-level test.

Mechanism taxonomy: carrier changed $\times$ reuse horizon

strict RTT is one operational cell, not a synonym for every form of test-time improvement

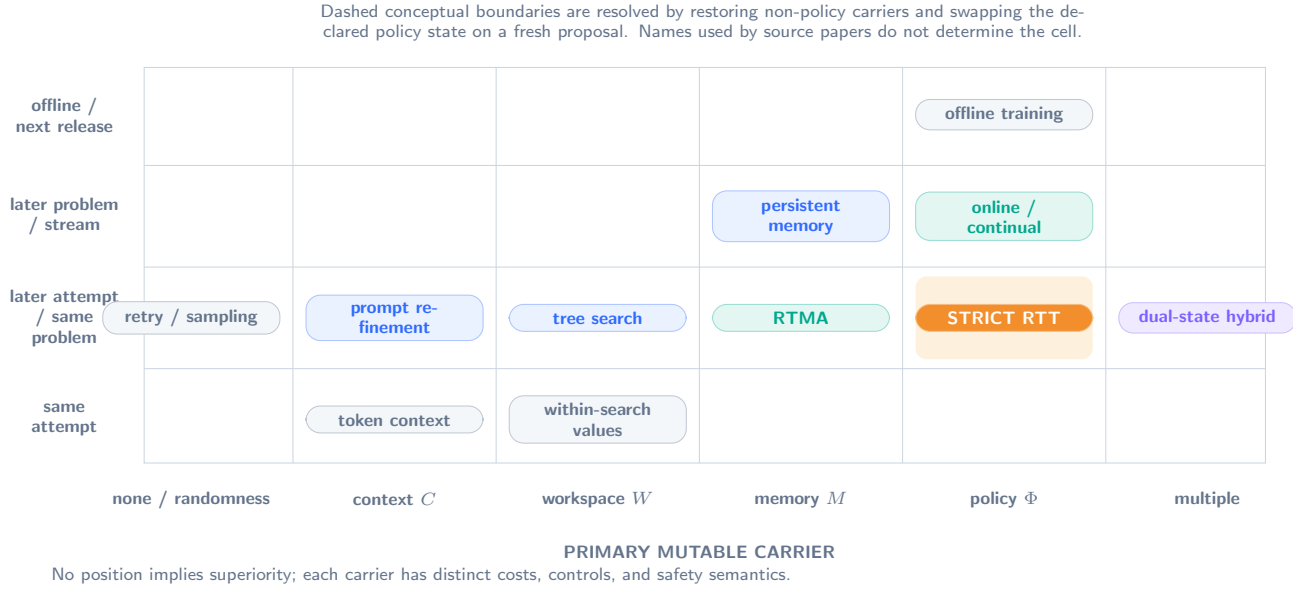


Figure 6: **RTT versus neighboring mechanisms (DEFINITION)**. Horizontal position records the carrier that changes; vertical position records reuse time. Strict RTT occupies the policy-state/same-unresolved-problem cell. Dashed borders indicate hybrids whose classification requires state-specific swaps; no cell is presented as universally superior.

4.2 No-read and fixed-policy results

Lemma 4.1 (Unused policy update cannot change the later law). *PROVED UNDER STATED ASSUMPTIONS* Fix an episode and suppose that, conditional on the non-policy state (X, C, M, W, Ξ, B) , every transition kernel from u_j through attempt $j+1$ is independent of Φ_{j+1} . Then replacing Φ_{j+1} by any other admissible state leaves the conditional law of the later attempt and its outcome unchanged.

Proof. Condition on the recorded non-policy state at u_j . The first post-update transition has the same kernel under every value of Φ_{j+1} by assumption. If the joint law through event n is the same, integrating the common event- $(n+1)$ kernel against that common law gives the same joint law through $n+1$. Induction continues through the finite attempt and its outcome. Thus the conditional laws coincide. Full details and the kernel factorization are restated in Appendix A. $\square$

The lemma distinguishes a state commit from actual reuse. A framework can spend substantial update compute and still have exactly zero causal path to later actions because the adapter was not mounted, the route bypassed it, or the state affected only a dead branch.

Proposition 4.2 (Fixed-policy search separation). *PROVED UNDER STATED ASSUMPTIONS* Consider any procedure that, during one episode, samples, scores, branches, backtracks, or votes using an action kernel with constant policy state $\Phi_j \equiv \phi_0$. If its only mutations are to (C, M, W, Ξ, B) , the procedure has no strict RTT witness, regardless of its compute budget or success rate.

Proof. W3 in Definition 3.1 requires a nontrivial committed policy-state transition, and W7 requires a later fresh-proposal law that differs under the post-update version. Constancy of Φ violates W3 directly. Changes to the other typed components do not repair that violation. The conclusion is classificatory, not a claim that search is ineffective. $\square$

Proposition 4.3 (Context- and memory-only separation). *PROVED UNDER STATED ASSUMPTIONS* If $C_{j+1} \neq C_j$ or $M_{j+1} \neq M_j$ while $\Phi_{j+1} = \Phi_j$, later behavior may change, but the trace is not strict RTT under Definition 3.1.

Proof. The policy-state transition condition W3 fails. Because Definition 3.3 holds C and M fixed when testing policy state, a retrieval- or prompt-mediated behavior change cannot substitute for W7. Such a trace should be reported as context refinement or memory adaptation, possibly RTMA, rather than erased from the analysis. $\square$

Corollary 4.4 (More inference compute is not sufficient). *PROVED UNDER STATED ASSUMPTIONS* Increasing the number of samples, tree nodes, tokens, tool calls, or retries is not sufficient for strict RTT.

Proof. Each increase can be implemented with constant Φ and mutations confined to W , C , or B . Proposition 4.2 applies. $\square$

4.3 Counterexamples to common classifications

Counterexample 4.5 (Lucky retry). **DEFINITION** A fixed stochastic policy fails once and succeeds on a second independent sample. The outcome changes, but there is no policy-state update. This is repeated inference, not strict RTT.

Counterexample 4.6 (Reflection in the prompt). **DEFINITION** A critique is appended to C_{j+1} and a frozen model retries. The later attempt is feedback-conditioned through context, not post-update-policy-conditioned. This is prompt/context refinement.

Counterexample 4.7 (Episodic memory write). **DEFINITION** The system writes “avoid API v1” to M and retrieves it on retry while weights and fast policy state remain unchanged. The mechanism is memory adaptation, not strict RTT.

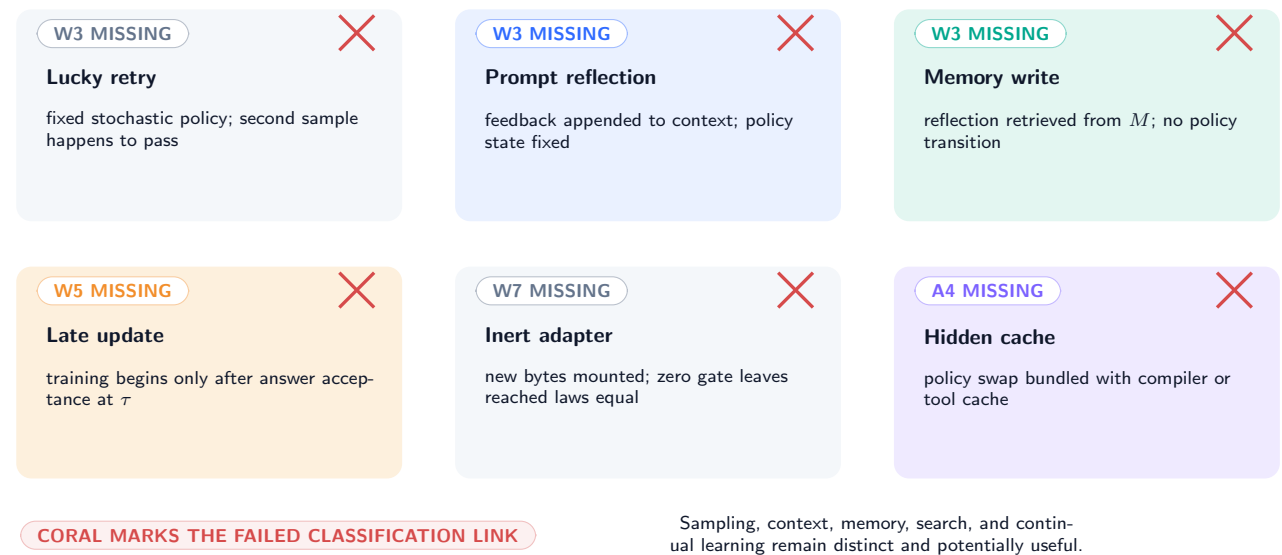
Counterexample 4.8 (Update after termination). **DEFINITION** The correct answer is accepted at τ , after which the model trains on the completed trace for future problems. That may be online or continual learning, but $u_j \geq \tau$ violates W5 for the completed problem.

Counterexample 4.9 (Mounted but inert adapter). **DEFINITION** A new adapter hash is mounted, but a zero gate makes every reached action distribution identical to the pre-update distribution. W6 holds while W7 fails; this is only a syntactic update–reuse trace.

Counterexample 4.10 (Hidden cross-arm cache). **DEFINITION** An apparent update benefit disappears when a compiler cache and random-number stream are restored. The original comparison bundled Φ with W and Ξ ; it did not identify a policy-state effect.

Counterexample gallery: better or different is not enough

each panel changes the later transcript while omitting or confounding one required causal link



Strict RTT requires one complete W1–W8 conjunction; success cannot compensate for a missing witness field.

Figure 7: **False-classification gallery (formal counterexamples)**. Each panel can produce a different or better later answer without satisfying all strict-witness conditions. Coral marks the single missing or confounded link; it does not mark the underlying method as scientifically failed.

4.4 Ambiguous recurrent and hypernetwork state

Some architectures blur conventional storage labels. A recurrent fast state that persists across attempt resets and directly parameterizes the fresh proposal kernel may be Φ even if it is not called a weight. A hypernetwork code that regenerates adapter weights is also policy state if the code swap changes the fresh proposal law. By contrast, a recurrent token cache rebuilt solely from the appended transcript is context state. The decisive audit is not the tensor’s name or device; it is the typed intervention with non-target carriers restored.

5 Identification, Intervention, and Impossibility

5.1 Transcripts do not identify the hidden mechanism

Theorem 5.1 (Finite-history emulation). *PROVED UNDER STATED ASSUMPTIONS* Let an adaptive controller induce a joint distribution over a finite set of observable within-episode histories and actions, possibly through an unobserved mutable policy state. There exists a single fixed history-dependent policy $q(a \mid h)$, with no mutable policy state, that induces the same joint distribution over those observable histories and actions under the same environment kernels.

Proof. For every observable history h with positive probability under the adaptive controller, define

$$q(a \mid h) := \mathbb{P}_{\text{ad}}(A = a \mid H = h), \quad (9)$$

where the right side marginalizes any hidden policy state and update randomness. Define q arbitrarily on zero-probability histories. At the first decision, q matches the adaptive conditional action law. If the joint observable law matches through a history h , the action conditional matches by construction and the next-observation conditional matches because the environment kernel is shared. Induction over the finite event horizon gives equality of the full observable joint law. The map q is fixed before the emulated episode; it conditions on history but does not update policy state. $\square$

Corollary 5.2 (Observational non-identifiability of strict RTT). *PROVED UNDER STATED ASSUMPTIONS* A finite transcript distribution, even one showing systematic improvement after feedback, cannot by itself identify a strict RTT mechanism.

Proof. Theorem 5.1 provides an observationally equivalent fixed-policy mechanism. Distinguishing the mechanisms therefore requires additional observables (state/read receipts) or interventions (state restoration/swaps). $\square$

The theorem is an existence result, not a claim that the emulator is compact or learnable. Its purpose is epistemic: behavioral logs cannot certify the hidden update type merely because a fixed lookup policy might be impractically large.

5.2 A randomized state–feedback factorial design

Let $R \in \{0, 1\}$ denote the policy-state deployment arm and $F \in \{0, 1\}$ the feedback-semantic arm. In all arms, attempt 1 and update compute are executed. When $F = 1$, the update receives the true registered feedback; when $F = 0$, it receives a prespecified within-block permutation or matched sham feedback. When $R = 1$, the resulting post-update state is mounted for the fresh retry; when $R = 0$, the registered pre-update state is restored. Context, memory, workspace, tool state, budgets, and randomization are restored or randomized identically. Let $Q(r, f) \in [0, 1]$ be the potential qualification outcome.

Define the true-feedback state effect and semantic interaction

$$\theta_{\text{state}} = \mathbb{E}[Q(1, 1) - Q(0, 1)], \quad (10)$$

$$\theta_{\text{int}} = \mathbb{E}[Q(1, 1) - Q(0, 1) - Q(1, 0) + Q(0, 0)]. \quad (11)$$

The first measures mounting the true-feedback update. The second asks whether that effect exceeds the effect of a compute-matched sham update. Neither estimand is automatically positive.

Theorem 5.3 (Unbiased factorial estimator). *PROVED UNDER STATED ASSUMPTIONS* Under Assumptions 2.6 and 2.7, independent randomized assignment of (R, F) makes each observed cell mean $\hat{\mu}_{r,f}$ unbiased for $\mathbb{E}[Q(r, f)]$. Consequently,

$$\hat{\theta}_{\text{int}} = \hat{\mu}_{11} - \hat{\mu}_{01} - \hat{\mu}_{10} + \hat{\mu}_{00} \quad (12)$$

is unbiased for θ_{int} , and $\hat{\mu}_{11} - \hat{\mu}_{01}$ is unbiased for θ_{state} .

Proof. By randomized assignment, (R, F) is independent of the vector of potential outcomes and has positive cell probabilities. Consistency gives $Q_{\text{obs}} = Q(R, F)$. Therefore $\mathbb{E}[Q_{\text{obs}} \mid R = r, F = f] = \mathbb{E}[Q(r, f)]$ for each cell. Taking the stated linear combinations and using linearity of expectation proves both claims. $\square$

Randomization does not repair a wrong outcome definition, a broken reset, or interference across episodes. It only identifies the displayed potential-outcome contrasts under the declared protocol.

Protocol 5.4 (Minimum strict-RTT qualification). *PLANNED TEST* Before measuring task success, a future implementation should preregister: (i) immutable episode IDs; (ii) a fresh-proposal snapshot; (iii) policy and non-policy state hashes; (iv) a balanced (R, F) assignment; (v) common-random-number or sufficiently replicated distribution probes; (vi) the total-variation lower bound and uncertainty rule for W7; and (vii) a reset-negative control. Failure to pass W1–W8 stops the benefit claim.

5.3 Reset isolation

Theorem 5.5 (Complete reset implies post-reset equality). *PROVED UNDER STATED ASSUMPTIONS* Suppose two arms are restored at time s to exactly the same complete state S_s in Equation (1), including external services and random-generator state, and thereafter use the same transition kernels. Then every post- s observable has the same conditional distribution in the two arms.

Proof. Conditional on the common S_s , the next-event kernels are identical. Applying the same induction over successive transition kernels as in Lemma 4.1 gives equality of all finite-dimensional post-reset distributions. $\square$

Corollary 5.6 (Reset-negative control). *PROVED UNDER STATED ASSUMPTIONS* Under Theorem 5.5’s assumptions, a systematic post-reset arm difference falsifies at least one reset, kernel-equality, or no-interference assumption; it cannot be attributed to the erased policy update.

The word “complete” is demanding. Files, retrieval indices, tool sessions, compiler caches, scheduler state, numerical nondeterminism, optimizer moments, and random streams all require an explicit disposition. A byte-equal model file is not a complete reset certificate.

5.4 Verifier uncertainty and reward circularity

Let $S \in \{0, 1\}$ denote true task success under a prespecified contract and $V \in \{0, 1\}$ a verifier decision.

Proposition 5.7 (Verifier alignment bound). *PROVED UNDER STATED ASSUMPTIONS* For any policy arm k ,

$$|\mathbb{E}[V_k] - \mathbb{E}[S_k]| \leq \mathbb{P}(V_k \neq S_k). \quad (13)$$

For two arms a, b with error rates at most $\varepsilon_a, \varepsilon_b$,

$$|\{\mathbb{E}[V_a] - \mathbb{E}[V_b]\} - \{\mathbb{E}[S_a] - \mathbb{E}[S_b]\}| \leq \varepsilon_a + \varepsilon_b. \quad (14)$$

Proof. Because S, V are binary, $|V - S| = \mathbf{1}\{V \neq S\}$. Thus $|\mathbb{E}[V - S]| \leq \mathbb{E}|V - S| = \mathbb{P}(V \neq S)$. Apply this once per arm and use the triangle inequality for the second claim. $\square$

Theorem 5.8 (Sign non-identifiability without a verifier link). *PROVED UNDER STATED ASSUMPTIONS* Suppose only transcripts, arm assignments, and a proxy verifier V are observed, while no assumption or audit links V to latent success S . Then the sign of a measured arm effect on S is not identified in general.

Proof. Fix any observed joint law of transcripts, arms, and V . One compatible latent model sets $S = V$; another sets $S = 1 - V$. Both preserve every observed distribution. Their arm effects satisfy $\mathbb{E}[S_a - S_b] = \mathbb{E}[V_a - V_b]$ and $\mathbb{E}[S_a - S_b] = -\mathbb{E}[V_a - V_b]$, respectively. Unless the proxy effect is zero, the signs oppose. Therefore observed data alone do not identify the true-success direction. $\square$

This impossibility is especially relevant when the same model family generates feedback, trains the update, and judges the final answer. Agreement inside that loop can increase while external correctness falls. A held-out executable contract, independent judge, or bounded verifier-error audit is not optional evidence decoration; it is part of the estimand.

6 Update Operators, Budgets, Lifetimes, and Objectives

6.1 RTT is not defined as gradient descent

Let $G_j = (X, Z_{\leq j}, Y_{\leq j}, C_j, M_j, W_j, \Xi_j, B_j)$ be the causally available update record. A general bounded update is

$$\Phi_{j+1} = \Pi_{\mathcal{K}(B_j)}(\mathcal{U}_\Theta(\Phi_j, G_j; R_j)), \quad (15)$$

where $\mathcal{U}_\Theta$ is an update operator, $\Pi_{\mathcal{K}(B_j)}$ enforces a budget- and safety-dependent admissible set, and Θ is an outer-trained rule or fixed algorithm. Table 4 lists nonexclusive realizations.

Operator family	Mutable policy state	Within-problem update	Distinct audit risk
Full/partial gradient	Selected weights and optimizer moments	One or more bounded gradient steps	Optimizer state, nondeterminism, support drift
Low-rank/adapters	Factor matrices or gates	Gradient, closed-form, or learned step	Mounted path may be gated off; rank/cost accounting
Learned optimizer	Optimizer recurrent state plus target parameters	Meta-learned map from gradients/feedback	Outer-training leakage and hidden lifetime
Recurrent/fast weights	Persistent policy-generating hidden state	State-transition rule from feedback	Confusion with ordinary token cache
Hypernetwork	Code or generator parameters	Feedback changes code/conditioning	Generated weights must be versioned and swapped
Bounded controller	Rules, logits, routing policy, or finite controller	Verified edit or constrained solver	Relabeling search workspace as policy

Table 4: **Update-operator taxonomy (DEFINITION)**. An operator enters strict RTT only through the same W1–W8 witness; its mathematical form does not confer status. LoRA is one possible low-rank substrate [1], not part of the definition.

The projection in Equation (15) can enforce norm, rank, sparsity, layer, latency, or memory limits. It does not itself guarantee useful learning. A learned optimizer may use an outer parameter Θ trained across tasks, while Φ is reset and adapted inside one episode. Their lifetimes and evidence must be separated: performance of Θ does not prove that a particular $\Phi_j \rightarrow \Phi_{j+1}$ was used.

6.2 Within-problem and outer objectives

For a fixed causal record at update time, a local update may target

$$J_j(\phi) = \mathbb{E}[R_{j+1} - \lambda_c C_{j+1} - \lambda_h H_{j+1} \mid \mathcal{F}_{u_j}, \text{do}(\Phi_{j+1} = \phi)], \quad (16)$$

where R_{j+1} is a prespecified retry utility, C_{j+1} is charged cost, and H_{j+1} is a harm variable. This conditional objective is not observable without assumptions or interventions; a proxy verifier defines a different objective unless its link to R is audited.

An outer design parameter Θ should be evaluated over complete episodes:

$$\mathcal{J}(\Theta) = \mathbb{E}_{X \sim \mathcal{D}} \left[U(X, Z_{1:J_\tau}) - \lambda_{\text{tok}} C_{\text{tok}} - \lambda_{\text{tool}} C_{\text{tool}} - \lambda_{\text{upd}} C_{\text{upd}} - \lambda_{\text{state}} C_{\text{state}} - \lambda_{\text{harm}} H - \lambda_{\text{drift}} D(\Phi_{\tau \wedge J}, \Phi_1) \right], \quad (17)$$

subject to hard per-episode budget, lifetime, verifier, and safety constraints. J_τ is the number of attempts begun before resolution. Reporting only final accuracy removes the defining cost of within-problem learning and can make an arbitrarily expensive method look dominant.

Definition 6.1 (Training-only realized-future teacher). **DEFINITION** An outer teacher may use completed future trajectories to estimate the counterfactual value of update actions during offline training. Any value, label, hyperparameter choice, or state version selected with post- u_j information is *training-only*. Deployment remains valid only if its committed update is $\mathcal{F}_{u_j}$ -measurable. The future teacher is not a deployed RTT witness.

6.3 Policy-gradient identity and its assumptions

Let a retry trajectory Z have density $p_\phi(z \mid \mathcal{G})$ conditional on a pre-retry record $\mathcal{G} \subseteq \mathcal{F}_{u_j}$, and let $R(Z)$ be integrable.

Proposition 6.2 (Conditional score-function identity). **PROVED UNDER STATED ASSUMPTIONS** Suppose p_ϕ is differentiable, its support does not depend on ϕ , differentiation and integration may be interchanged, and $b(\mathcal{G})$ is any integrable baseline independent of the sampled retry actions conditional on $\mathcal{G}$. Then

$$\nabla_\phi \mathbb{E}[R(Z) \mid \mathcal{G}] = \mathbb{E}[(R(Z) - b(\mathcal{G})) \nabla_\phi \log p_\phi(Z \mid \mathcal{G}) \mid \mathcal{G}]. \quad (18)$$

Proof. Using the stated interchange, $\nabla_\phi \int R(z) p_\phi(z \mid \mathcal{G}) dz = \int R(z) p_\phi(z \mid \mathcal{G}) \nabla_\phi \log p_\phi(z \mid \mathcal{G}) dz$. The baseline term has conditional expectation $b(\mathcal{G}) \int \nabla_\phi p_\phi(z \mid \mathcal{G}) dz = b(\mathcal{G}) \nabla_\phi 1 = 0$. Subtracting it yields Equation (18). $\square$

The proposition proves an identity, not low variance, verifier validity, safe optimization, or improvement after a finite step. If support changes under decoding truncation or tool constraints, the stated form may fail.

Lemma 6.3 (Group-relative degeneracy). *PROVED UNDER STATED ASSUMPTIONS* Consider a group update whose per-sample weight is

$$A_i = \frac{R_i - \bar{R}}{s_R + \varepsilon}, \quad \varepsilon > 0. \quad (19)$$

If $R_1 = \dots = R_K$, then every $A_i = 0$ and any update linear in the A_i is exactly zero.

Proof. Equal rewards imply $R_i = \bar{R}$ for every i , so every numerator vanishes. A linear combination of score terms with zero coefficients is zero. $\square$

This is a within-group credit result, not a statement that all group-relative algorithms always fail. It yields a concrete stop condition: if the verifier collapses to one value within update groups, more optimizer steps cannot recover a signal from Equation (19).

6.4 Budget and lifetime ledgers

Let the remaining budget be a vector

$$B_j = (b_j^{\text{tok}}, b_j^{\text{tool}}, b_j^{\text{sec}}, b_j^{\text{upd}}, b_j^{\text{byte}}, b_j^{\text{energy}}), \quad (20)$$

where energy may be an explicitly labeled proxy if direct measurement is unavailable. Every attempt/update pair incurs a nonnegative cost vector c_j .

Proposition 6.4 (Hard-ledger feasibility). *PROVED UNDER STATED ASSUMPTIONS* Suppose an operation is admitted only when $c_j \leq B_j$ componentwise and then sets $B_{j+1} = B_j - c_j$. If $B_1 \geq 0$, every admitted prefix has $B_j \geq 0$ and cumulative cost at most B_1 .

Proof. Componentwise subtraction preserves nonnegativity by the admission rule. Telescoping gives $\sum_{i < j} c_i = B_1 - B_j \leq B_1$. $\square$

Lifetime class	Required reset/transfer statement	Scientific interpretation
Attempt-local	State expires before another attempt	Not a cross-attempt RTT carrier
Problem-local	State survives retries and expires at τ	Canonical strict RTT lifetime
Session-local	State may cross problem IDs	Mixture with online/continual adaptation
Persistent	State enters later sessions/releases	Continual/offline learning artifact; separate governance

Table 5: **Policy-state lifetime classes** (DEFINITION). Strict RTT concerns the in-problem update–reuse event; cross-problem retention is an additional mechanism that must not be hidden.

An episode-local adapter may still leak through optimizer caches, compilation artifacts, retrieval indices, or scheduler state after its nominal deletion. Lifetime certification therefore concerns the complete causal state, not only the primary tensor file.

7 Stability, Rollback, and Stop Conditions

7.1 Local drift bounds are conditional, not global safety proofs

For a fixed admissible history h , write $\pi_i(\cdot \mid h)$ for the fresh-proposal law after update i .

Theorem 7.1 (Cumulative local drift). *PROVED UNDER STATED ASSUMPTIONS* Suppose for $i = 1, \dots, K$ and a fixed history h ,

$$\text{KL}(\pi_i(\cdot \mid h) \parallel \pi_{i-1}(\cdot \mid h)) \leq \kappa_i(h) < \infty. \quad (21)$$

Then

$$\text{TV}(\pi_K(\cdot \mid h), \pi_0(\cdot \mid h)) \leq \sum_{i=1}^K \sqrt{\frac{\kappa_i(h)}{2}}. \quad (22)$$

For any measurable $g : \mathcal{A} \rightarrow [0, 1]$,

$$|\mathbb{E}_{\pi_K}[g(A) \mid h] - \mathbb{E}_{\pi_0}[g(A) \mid h]| \leq \min \left\{ 1, \sum_{i=1}^K \sqrt{\frac{\kappa_i(h)}{2}} \right\}. \quad (23)$$

Proof. Pinsker’s inequality bounds each adjacent total variation distance by $\sqrt{\kappa_i(h)}/2$. The triangle inequality for total variation gives Equation (22). The difference in expectations of a $[0, 1]$ function is at most total variation; total variation is at most one, yielding Equation (23). $\square$

The theorem is pointwise in h . A KL constraint measured on update trajectories does not automatically control unseen histories, tool calls, or external side effects. Nor does small local drift imply positive utility: a small harmful change is still harmful.

Theorem 7.2 (No nontrivial update is uniformly improving). *PROVED UNDER STATED ASSUMPTIONS* Let π and π' be distinct probability measures over actions at some fixed history. There exists a bounded utility $g : \mathcal{A} \rightarrow [0, 1]$ such that $\mathbb{E}_{\pi'} g < \mathbb{E}_{\pi} g$.

Proof. Because $\pi' \neq \pi$, there is a measurable set A on which their probabilities differ. If $\pi'(A) < \pi(A)$, choose $g = \mathbf{1}_A$. Otherwise the complement satisfies $\pi'(A^c) < \pi(A^c)$, so choose $g = \mathbf{1}_{A^c}$. In either case the expected utility decreases. $\square$

This simple impossibility explains why a universal “safe learning rate” cannot follow from nontriviality alone. Safety requires a specified utility/harm family, coverage assumptions, conservative gates, and rollback; it cannot mean improvement for every possible downstream objective.

7.2 Transactional update contract

Definition 7.3 (Rollback-capable policy-state transaction). **DEFINITION** A policy update transaction contains: (i) an immutable pre-state snapshot; (ii) a candidate state and complete auxiliary state; (iii) typed validation receipts; (iv) canary probes on registered histories; (v) an atomic mount or NULL decision; (vi) post-mount monitoring; (vii) a bounded-time rollback path restoring the complete pre-state; and (viii) quarantine of rejected candidate artifacts. Partial mounts do not count as commits.

Gate	Prespecified check	Mandatory action on failure
Causal	Update inputs are $\mathcal{F}_{u_j}$ -measurable	Quarantine; label leakage
Integrity	Hashes, shapes, optimizer state, route, and version agree	Atomic NULL; no retry mount
Budget	Worst-case update/retry cost fits remaining ledger	Skip update or stop episode
Functional	Fresh-proposal W7 lower bound clears threshold	Label inert; no RTT claim
Local stability	KL/norm/logit and tool-policy deltas under caps	Roll back candidate
Canary safety	No registered severe-harm event; harm CI below bound	Roll back and stop branch
Verifier	Error/abstention and group diversity within gate	Do not optimize proxy
Reset control	Negative-control arms exchangeable within tolerance	Stop causal attribution

Table 6: **Transactional qualification and stop rules (DESIGN PRESCRIPTION)**. Thresholds must be registered before seeing target-arm outcomes.

7.3 What rollback can and cannot undo

Rollback can restore a declared digital state. It cannot automatically undo an external side effect already caused by a tool call, disclosure, transaction, or human decision. A safe episode therefore separates *proposal actions* from *irreversible actions*. Candidate policy versions may generate proposals in a sandbox; an independently governed executor decides whether to commit side effects. If the task necessarily includes irreversible actions, the harm estimand and authorization boundary must include them before the update is mounted.

Definition 7.4 (Stop condition). **DEFINITION** A stop condition is a precommitted predicate on causally available audit variables whose truth forces one of: skip update, return NULL, roll back, abandon the episode, or escalate to external review. A stop condition is invalid if it is relaxed after observing an unfavorable target-arm result without recording a new protocol.

Minimum stops include: future leakage; missing state/read receipts; zero functional divergence; collapsed within-group feedback; failed complete-reset control; verifier error exceeding its bound; safety-canary breach; budget overrun; nonfinite update; version mismatch; and inability to restore the full pre-state. An episode may stop even when the current candidate would have solved the problem; evidence discipline takes precedence over retrospective success.

7.4 Stability across repeated within-problem updates

Repeated updates create a feedback loop: the current policy determines which attempts are seen, those attempts determine feedback, and feedback changes the next policy. Three ledgers should therefore be distinct:

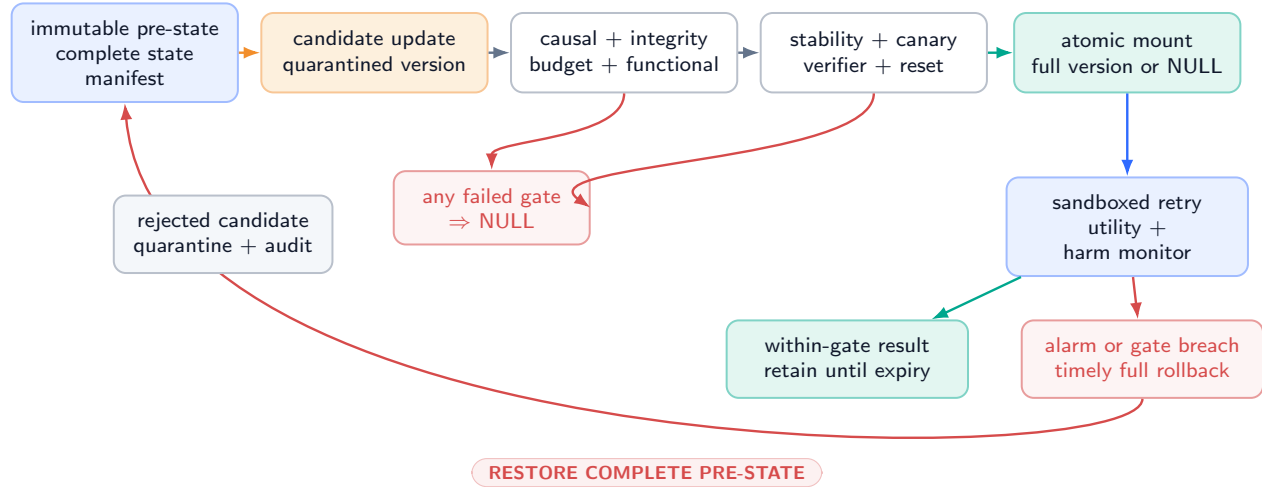
1. **Mechanism ledger:** hashes, update inputs, W1–W8, and state-swap divergence for each transition;

2. **Utility ledger:** true/proxy outcomes, verifier uncertainty, and prespecified episode utility;
3. **Risk ledger:** local drift, canary harms, side effects, rollback latency, and residual state after expiry.

A positive utility ledger cannot repair a missing mechanism witness, and a valid mechanism witness cannot waive a risk failure.

Transactional policy update: qualification, mount, monitor, rollback

a candidate remains quarantined until every registered gate passes; failures return NULL or restore the immutable snapshot



Rollback cannot undo an external side effect already committed. Irreversible effects require separate executor governance and authorization.

DESIGN PRESCRIPTION / UNVALIDATED: this path is a required contract, not an implemented or tested safety system.

Figure 8: **Safety, rollback, and stop-condition path (DESIGN PRESCRIPTION)**. A candidate update remains quarantined until causal, integrity, budget, functional, stability, canary, verifier, and reset gates pass. Any bounded coral failure returns NULL or restores the immutable snapshot. Deployment of a candidate is atomic; the figure reports a contract, not an implemented system.

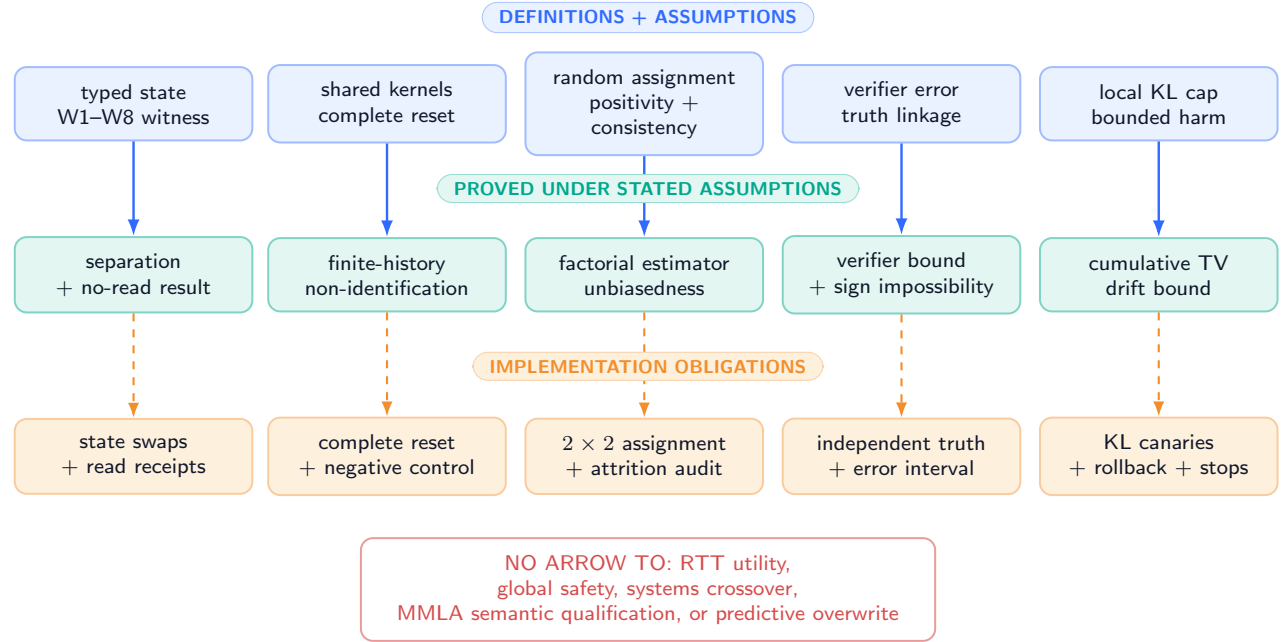
8 From Theorems to Implementation Obligations

8.1 Proof dependency map

The formal claims are shallow by design: each conclusion depends on a small, inspectable assumption set. Figure 9 and Table 7 prevent a common category error—implementing the conclusion’s name while omitting the assumptions that make it true.

Proof and assumption dependency map

selected premise bundles end in conditional results and implementation receipts; theorem text controls the full dependencies



The theorem-to-obligation table states the exact premise receipt required for every formal result.

Figure 9: **Assumption and proof dependency map (PROVED UNDER STATED ASSUMPTIONS for the drawn logical arrows).** Blue nodes are definitions/assumptions, teal nodes are proved statements, and amber nodes are design consequences. No arrow enters an empirical-success node because this paper contains none. Appendix A provides the expanded derivations.

Table 7: **Theorem-to-implementation obligations.** Formal results become usable only when their premise receipts exist.

Formal statement	Essential assumptions	Required implementation evidence	Does not establish
Lemma 4.1	Conditional independence of every later kernel from Φ	Route/read traces; state-swap or route-disable probe	Update usefulness
Proposition 4.2	Stable typed state partition	Fresh-proposal workspace inventory	Search inferiority
Theorem 5.1	Finite observable horizon; same environment kernels	Transcript schema and hidden-state inventory	Efficient fixed emulator
Theorem 5.3	Randomization, positivity, consistency, no bundled carrier changes	Assignment receipt; arm counts; restore hashes; attrition audit	Correct outcome/verifier
Theorem 5.5	Complete state equality and common kernels after reset	Full-state manifest, external-service reset, seed receipt	That any practical reset is complete
Proposition 5.7	Binary success and measured verifier error	Independent adjudication sample and error CI	Small error off the audited support
Theorem 5.8	No constraint linking verifier to truth	Explicit absence/presence of external truth contract	That the proxy is actually adversarial
Proposition 6.2	Differentiability, fixed support, integrability, legal interchange	Decoder support audit; finite-gradient checks	Low variance or finite-step gain
Lemma 6.3	Centered group-relative weights; linearity in advantages	Per-group rewards and computed advantages	Failure under noncollapsed rewards

Formal statement	Essential assumptions	Required implementation	evidence	Does not establish
Proposition 6.4	Nonnegative charged costs; hard admission rule	Monotone signed resource ledger		Accuracy or efficiency dominance
Theorem 7.1	Per-history finite adjacent KL	Common-history log-probability probes		Global behavioral safety
Theorem 7.2	Nonidentical action laws; arbitrary bounded utilities	Utility/harm family stated before update		Harm on the chosen real task

8.2 Minimum trace schema

Definition 8.1 (RTT trace bundle). **DEFINITION** A qualification bundle contains one immutable manifest linking:

1. episode/problem/version identifiers and complete problem contract;
2. attempt start/end events, observations, actions, tool calls, and randomization receipts;
3. feedback bytes, producer identity, availability time, and truth/proxy status;
4. pre/candidate/post policy-state bytes or content-addressed references, plus auxiliary optimizer/router state;
5. context, memory, workspace, external-service, and budget manifests at each restore boundary;
6. update inputs, code/config hash, numerical mode, cost vector, candidate validation, commit/NULL decision;
7. later policy-kernel read trace and fresh-proposal state-swap outputs;
8. outcome, verifier, safety, rollback, expiry, and residual-state receipts.

Every omission is a declared unknown, not an implicitly controlled variable.

8.3 Hidden-state and future-leakage audit

The following channels require explicit disposition before causal attribution:

- model weights, adapters, gates, normalization statistics, optimizer moments, activation/attention/KV caches, recurrent state, quantization scales, compiler graphs, and numerical kernels;
- prompts, system messages, hidden chain-of-thought carriers, retrieved documents, episodic/semantic memory, search trees, scratch files, databases, environment variables, tool sessions, and service-side caches;
- random-number generators at every library/device, scheduler order, batching neighbors, asynchronous callbacks, and retry policies;
- verifier models, reference answers, unit tests, search or tool results, and any post-update outcome used to choose checkpoints or thresholds.

Protocol 8.2 (Leakage challenge). **PLANNED TEST** Construct a canary field revealed strictly after update time and guaranteed irrelevant to the problem. Search every update input, serialized state, log, and selection decision for dependence on that field; additionally train a held-out decoder to predict the canary from the committed policy-state delta. Any above-chance result beyond the registered tolerance stops the causal claim until the channel is explained. A negative canary test is evidence about tested channels only.

8.4 Circularity audit

Four loops are especially dangerous:

1. **Self-verification:** the policy family produces answers and labels them;
2. **Selection on target outcomes:** learning rates, checkpoints, or stop thresholds are chosen using the same episodes used for the claim;
3. **Adaptive exclusion:** difficult failures are discarded after arm assignment;
4. **Mechanism by outcome:** only successful retries are inspected for update receipts.

The remedy is not to ban internal feedback. It is to define whether it is a proxy, bound or externally audit its error, freeze selection rules on a disjoint split, report intention-to-treat and attrition, and inspect mechanism receipts for every assigned episode. Appendix C gives a printable audit matrix.

8.5 Reproducible state swaps

A state swap must be semantically complete. For an adapter, this includes factor matrices, scaling, gates, routing tables, normalization state, optimizer-coupled forward state, dtype, and mount location. The pre- and post-state should be evaluated on the same registered histories under common random numbers where possible. For sampled text, a lower bound on total variation may be estimated from repeated draws with simultaneous confidence intervals, but a failure to reject equality is not proof of equality. Exact logit comparison is preferable when the action space and inference stack permit it.

9 Relationship to Adjacent Work

This section classifies *described protocols*, not communities or papers as wholes. A paper may contain several mechanisms, and a future implementation may differ from its prose. Strict RTT status belongs to a concrete trace satisfying Definition 3.1.

9.1 Test-time training and adaptation

Classical test-time training updates model parameters on an auxiliary self-supervised objective derived from an unlabeled test sample before the main prediction [7]. Fully test-time adaptation such as Tent updates normalization parameters online by entropy minimization on test batches [8]. These works establish the broader idea that parameters can change during evaluation. Their unit and timing need not match strict RTT: an auxiliary update before a first task attempt, or an update accumulated across a stream and used on later samples, is test-time adaptation without necessarily containing a feedback–update–retry witness inside one still-unresolved reasoning problem.

TTRL performs reinforcement learning on unlabeled test data using consensus-derived rewards and online parameter updates [13]. At the dataset level it is test-time reinforcement learning. Whether a particular item is strict RTT depends on whether feedback from that item updates policy state before the item ends and a later attempt on that same item uses the update; dataset-level improvement cannot substitute for that trace.

9.2 Inference-time scaling, search, and refinement

Self-consistency samples multiple reasoning paths and aggregates answers [9]. Tree of Thoughts explicitly searches over intermediate reasoning states with evaluation and backtracking [10]. ReAct interleaves reasoning traces with actions and tool observations [11]. These mechanisms enlarge computation, workspace, or context and can substantially change outcomes while keeping the underlying proposal policy fixed. Under our state partition they are not strict RTT unless an additional policy-state update passes the fresh-proposal test.

Self-Refine iteratively produces feedback and revised outputs without supervised training, additional training, or reinforcement learning [5]. Reflexion explicitly reinforces agents through linguistic feedback and an episodic memory buffer rather than weight updates [6]. They are canonical reasons to preserve context/memory categories: trial-to-trial learning-like behavior need not be policy training. The label RTMA is useful for ordinary reasoning-time memory adaptation, but this paper makes no claim that it is the original name for such mechanisms.

9.3 Within-instance policy evolution

Policy of Thoughts describes within-instance online optimization: execution feedback drives GRPO updates to a transient LoRA adapter, which is reused during subsequent search and discarded after the problem [2]. StarOR describes node-level GRPO updates to a transient LoRA adapter maintained for the current optimization-modeling instance and reset for each new problem [4]. Their primary sources therefore describe candidate strict-RTT architectures more closely than fixed-policy search. This paper does not revalidate their experiments or infer W1–W8 from descriptions alone. In particular, an independent strict audit would still need exact update times, unresolved status, actual mounted-state reads, fresh-proposal divergence, reset completeness, budgets, and verifier provenance.

9.4 Continual learning, low-rank state, and memory

Continual learning studies policy retention across sequences of tasks and the prevention of catastrophic forgetting; elastic weight consolidation is a representative example [3]. Its primary reuse horizon is later tasks, whereas strict RTT requires reuse before the current problem ends. The regimes can be nested: a strict within-problem adapter may be reset at each endpoint while an outer continual learner changes the initialization across episodes.

LoRA provides a parameter-efficient low-rank adaptation substrate [1]. A LoRA update may be offline, continual, test-time, or strict RTT depending on timing and reuse; the matrix factorization alone determines none of those regimes.

MMLA formalizes bounded authoritative memory, causal event-conditioned writes, and training-only future valuation while reporting a current semantic-interface blocker [12]. Its public evidence does not validate RTT, and this theory paper does not validate MMLA predictive overwrite. A combined system would need independent policy-state and memory-state interventions as in Figure 4.

Protocol family	Described changing carrier	Typical reuse unit	Trace-level RTT interpretation
TTT / Tent [7, 8]	Model/statistical parameters	Current sample, batch, stream	Adjacent; inspect whether same-problem feedback precedes an actual retry.
Self-consistency / ToT [9, 10]	Samples/search workspace	Same problem	Fixed-policy form is not strict RTT.
ReAct / Self-Refine [11, 5]	Context, tool state, revision text	Same problem	Context/workspace adaptation unless policy state also changes.
Reflexion [6]	Episodic linguistic memory	Later trials	Memory adaptation; source explicitly contrasts with weight updates.
TTRL [13]	Policy parameters	Test data stream	Per-item strict status requires an item-level update–reuse trace.
PoT [2]	Per-instance transient LoRA	Subsequent same-instance search	Candidate strict RTT; W1–W8 not independently audited here.
StarOR [4]	Per-instance transient LoRA	Later nodes/iterations on same instance	Candidate strict RTT; W1–W8 not independently audited here.
Continual learning [3]	Persistent parameters	Later tasks	Different primary horizon; may contain nested RTT.
MMLA [12]	Bounded external/resident memory	Later tokens/segments	Memory program, not RTT evidence.

Table 8: **Related-work protocol map (source-verified, not a replication).** The final column applies this paper’s operational boundary cautiously and makes no priority or empirical-ranking claim.

The literature suggests a continuum rather than a winner: compute can be spent on broader exploration, richer nonparametric state, faster parametric adaptation, or combinations. The scientific task is to identify which carrier caused which effect at what cost, not to collapse all adaptation into one label.

10 A Future Qualification and Falsification Program

10.1 Stage gates

The program begins with mechanism identity and stops on failure. It does not begin with benchmark accuracy. Figure 10 shows the dependency order; Table 9 makes each rung falsifiable.

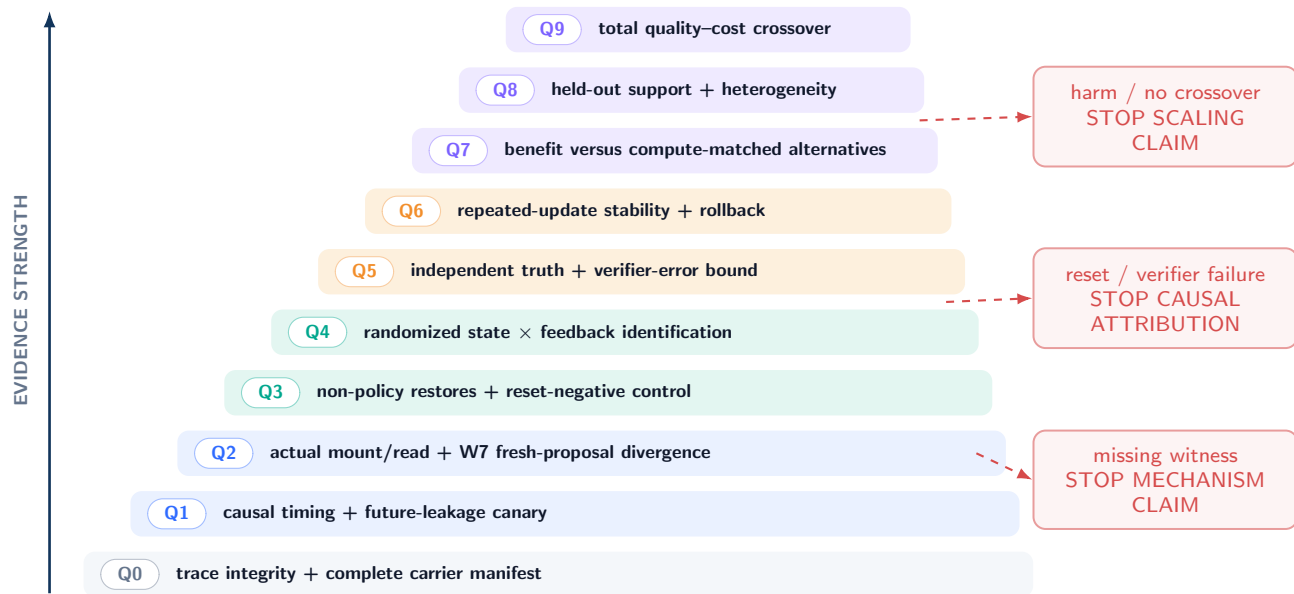
The gates are sequential because later quantities are uninterpretable when earlier premises fail. A utility difference cannot establish policy-state use if the declared carrier was never read, and a carrier-swap contrast cannot isolate policy state if a tool cache, context suffix, random-number stream, or workspace also changed. Likewise, an apparently favorable verifier score does not identify true utility until the verifier–truth error relation is bounded on the relevant arms. Passing a gate therefore means only that its registered receipts survived their own audit. It neither repairs a failed lower gate nor supplies evidence for a higher one.

Each gate must specify its analysis population before execution. The intention-to-treat population contains every assigned episode, including update crashes, refusals, timeouts, rollback events, and missing receipts. A separately reported qualification population may restrict attention to episodes whose W1–W8 records are complete, but that restriction is descriptive unless the selection mechanism is itself identified. Attrition, verifier abstention, and failed mounts are outcomes, not preprocessing inconveniences. Negative controls and deliberately corrupted traces belong in the same ledger so that an auditor can estimate whether the qualification machinery rejects mechanisms it was designed to reject.

Thresholds, budgets, and stop decisions must be frozen before outcome inspection: the minimum W7 divergence, verifier-error tolerance, local drift cap, severe-harm definition, equal-compute ledger, and minimum worthwhile utility gain. A negative terminal is a valid result at its gate. It should preserve the failed receipt, assigned arm, and available downstream measurements while withholding the claim that depended on the failure. This convention separates scientific falsification from system promotion: an implementation can be informative even when it never becomes eligible for scaling.

Future RTT qualification ladder: mechanism before benefit

PLANNED TESTS / NO RUNG EXECUTED advance only when the lower gate and its receipts survive



A clean negative terminal is admitted evidence about that gate; it does not authorize skipping upward.

Figure 10: **Future empirical qualification ladder (PLANNED TEST)**. Evidence may advance only upward: integrity and timing precede functional state use; identification precedes benefit; safety precedes scale; equal-compute crossover is last. Coral side arrows are registered stops. No rung has been executed or passed in this paper.

Table 9: **Planned qualification tests and stop rules (PLANNED TEST)**.

Gate	Question	Minimal protocol	Stop condition
Q0	Is the trace complete?	Hash code/config/state; immutable episode IDs; complete carrier and cost inventory.	Missing or inconsistent receipt.
Q1	Did timing satisfy W1–W5?	Event-clock audit against filtration and stopping time; future canary.	Update after resolution or detectable future dependence.
Q2	Was post-state actually used?	Mount/read trace plus exact or replicated fresh-proposal swap.	W6 absent or W7 lower bound ≤ 0 .
Q3	Are non-policy carriers isolated?	Restore context, memory, workspace, tools, RNG; separate carrier swaps.	Reset-negative control differs.
Q4	Is feedback semantic?	Randomized 2×2 state/feedback factorial; intention-to-treat analysis.	Assignment, positivity, attrition, or sham-compute failure.
Q5	Is the verifier linked to truth?	Independent adjudication, abstention and error bounds, adversarial cases.	Error bound too wide for claimed effect or group signal collapse.
Q6	Is repeated updating stable?	Registered KL/norm/canary caps; atomic rollback; residual-state audit.	Severe harm, nonfinite update, or rollback failure.
Q7	Does it help under matched compute?	Compare fixed-policy search, context, memory, sham update, and RTT under equal ledgers.	Confidence interval excludes the prespecified minimum gain in the wrong direction.
Q8	Does it transfer within declared support?	Held-out problem families, verifiers, seeds, and update counts; report heterogeneity.	Benefit confined to tuned items or hidden carryover.
Q9	Is total quality–cost favorable?	End-to-end wall time, tokens, tool calls, update compute, retained bytes, harm and failure cost.	No registered crossover or budget breach.

10.2 White-box mechanism benchmark

Protocol 10.1 (RTT-M0 mechanism falsifier). **PLANNED TEST** Construct a synthetic family with a hidden per-instance rule that cannot be identified from the initial prompt alone. Attempt 1 reveals a deterministic verifier bit about one proposed rule; an update may change only a small registered policy state; attempt 2 begins from a restored context/memory/workspace snapshot. Include (a) true update, (b) pre-state restore, (c) sham-feedback update, and (d) search-only workspace arms. The primary endpoint is W7 fresh-proposal divergence in the registered direction, not final accuracy. Reject mechanism qualification if any state carrier outside Φ differs across the required swap.

The benchmark should contain negative controls where the feedback is uninformative, the update route is gated off, the adapter is byte-different but functionally null, and the apparent rule is recoverable from a hidden cache. These cases test the auditor rather than the learner.

10.3 Executable reasoning benchmark

Protocol 10.2 (RTT-M1 executable retry). **PLANNED TEST** Use generated programs with isolated deterministic unit tests unavailable to the policy except through a bounded first-attempt feedback record. Freeze the task generator and hidden tests before method development. Preregister one retry, one bounded update, one complete restore, the 2×2 factorial, verifier-error spot checks, update/cost caps, and a severe-side-effect sandbox. Report every assigned episode and distinguish syntax, hidden-test truth, policy-state divergence, and final utility.

Compiler feedback is not automatically ground truth: passing public tests can overfit, and test-generation bugs can reward incorrect behavior. Hidden-test construction, contamination, and false-pass/false-fail audits remain part of Q5.

10.4 Repeated-update stress test

Protocol 10.3 (RTT-M2 repeated-update stability). **PLANNED TEST** On episodes requiring up to K retries, register a per-update KL cap κ_i , cumulative bound from Theorem 7.1, independent anchor histories, tool-action canaries, rollback latency, and a hard maximum update count. Randomize rollback challenges by injecting known-bad candidate states. Stop the full arm if any irreversible side effect, unbounded ledger, or failed complete-state restoration occurs.

The purpose is not to prove global safety. It is to falsify specific local contracts under adversarial but declared conditions and to measure how often the theoretical premises hold.

10.5 Equal-compute comparisons

The final comparison should allocate the same multidimensional budget, not merely the same number of sampled answers. Search may use its budget for more rollouts; context refinement for longer critique/revision; memory adaptation for writes and retrieval; RTT for update, state storage, probes, and retry. Wall-clock parallelism, accelerator occupancy, tool latency, failure recovery, and verifier calls must be included. A method that wins only after omitting update or safety costs has not established a quality–cost crossover.

Primary reporting should include:

- intention-to-treat and per-protocol estimates with arm counts and attrition;
- $\theta_{\text{state}}, \theta_{\text{int}}, \text{W7 divergence, verifier-error bounds, and true utility as distinct endpoints;}$
- distributions over problem families and seeds, not only pooled means;
- compute, time, retained-state, and side-effect ledgers with uncertainty;
- all preregistered stops, including negative and null terminals.

No successful result is predicted by this ladder. A clean failure at Q2 or Q4 would be scientifically useful: it would show that an implementation marketed as adaptive did not create or use the claimed policy-state path.

11 Discussion, Limitations, and Conclusion

11.1 Why adopt a strict definition?

A broad definition in which any feedback-conditioned retry counts as training would be easy to satisfy but difficult to falsify. It would merge sampling, search, prompt revision, memory, tool state, and policy adaptation even though they have different costs, safety properties, and scientific controls. The strict definition instead reserves one term for a specific causal pattern while leaving neighboring mechanisms first-class. A hybrid system can still be studied; it simply cannot attribute a bundled effect to policy learning without state-specific interventions.

The boundary is partly conventional. One could define “policy” to include the entire search tree or prompt, making every stateful computation a policy update. Definition 3.3 rejects that collapse for this paper by holding non-policy carriers fixed and testing fresh proposal laws. This choice is useful because it yields operational separation, but it is not a theorem that every community must adopt the same vocabulary.

11.2 Limitations of the framework

First, exact state interventions may be infeasible in opaque hosted systems. There, strict qualification may remain unidentifiable; behavioral improvement should be reported without a mechanism claim. Second, total variation at token-level histories can be expensive to estimate in large action spaces. Registered lower bounds or exact logit probes cover only tested histories. Third, the filtration assumes meaningful event timestamps; asynchronous tools and distributed updates require a partial-order extension. Fourth, potential-outcome consistency may fail when shared services create interference across arms. Fifth, a binary resolution time simplifies tasks with graded or revisable success. Sixth, the framework treats policy and memory as typed interfaces, but architectures with fully entangled fast state may resist clean partitioning. Such systems should report an inseparable bundled carrier rather than force a false decomposition.

The mathematical results are deliberately modest. The finite-history emulator may be exponentially large. The factorial theorem does not solve external validity. The verifier bound is only as useful as the audited error rate. The drift bound is local and can become vacuous after many updates. The no-uniform-improvement theorem does not imply that safe gains are impossible on a restricted task distribution. None of these results predicts that a practical optimizer will learn within an episode.

11.3 Boundary with MMLA evidence

This paper inherits the public author identity, style, and claim grammar of the MMLA V3 line, not an empirical success claim. The complete technical-report evidence record localizes a semantic-interface failure and keeps learned predictive overwrite closed. No MMLA result is used to claim that policy state can be updated and reused during reasoning. Conversely, this paper’s definitions do not repair or validate the MMLA memory interface. A future combined MMLA+RTT system would require four independent questions: did policy state change, did memory state change, did each state affect the appropriate fresh/retrieval path, and did either cause verified benefit under matched cost?

11.4 Conclusion

RTT is most useful as a falsifiable trace property: feedback from an attempt on one unresolved problem changes a declared functional policy state, the change commits before resolution, and a later attempt on that same problem actually uses it. This temporal and typed definition separates policy learning from spending more inference compute or carrying more nonparametric state. The accompanying theorems show why logs and proxy success are insufficient, while the qualification ladder states what future evidence would have to survive.

The paper’s strongest conclusion is therefore negative in form: neither a method name, an optimizer call, a different retry, nor a proof can establish strict RTT benefit. Establishing it requires state identity, causal timing, functional reuse, interventions, verifier alignment, budget accounting, and rollback evidence together. Those obligations are demanding by design. If future systems pass them, “learning before a single problem ends” will denote an identified mechanism rather than a metaphor.

References

- [1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [2] Zhengbo Jiao, Hongyu Xian, Qinglong Wang, Yunpu Ma, Zhebo Wang, Zifan Zhang, Dezhong Kong, and Meng Han. Policy of thoughts: Scaling test-time training for LLM reasoning via online policy evolution, 2026. Version 2, preprint.
- [3] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharmashan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [4] Jiajun Li, Yu Ding, Shisi Guan, Ran Hou, and Wanyuan Wang. StarOR: Synergizing tree search and test-time reinforcement learning for optimization modeling, 2026. Version 2, preprint.
- [5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.

- [6] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023.
- [7] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248. PMLR, 2020.
- [8] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.
- [9] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.
- [10] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [12] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future, 2026. Version 3.
- [13] Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning, 2025.

A Expanded Proofs

This appendix expands every numbered formal result. All statements retain the status **PROVED UNDER STATED ASSUMPTIONS**. The proofs concern the abstract model only and create no implementation or empirical evidence.

A.1 Kernel notation

Let E_n denote the complete event at clock n and $H_n = (E_0, \dots, E_n)$ its observable history. Conditional on a complete state and intervention arm, write $K_{n+1}(de \mid h_n)$ for the next-event transition kernel. For a finite horizon N , the joint law factors as

$$P(de_{0:N}) = P_0(de_0) \prod_{n=0}^{N-1} K_{n+1}(de_{n+1} \mid e_{0:n}). \quad (24)$$

All induction arguments below are applications of this factorization.

A.2 No-read and separation

Lemma 4.1. Condition on the non-policy state $N_{u_j} := (X, C, M, W, \Xi, B)$ at update completion. Let P^ϕ and $P^{\phi'}$ be the post-update laws under two mounted policy states. By hypothesis, for every reachable history and each later event,

$$K_{n+1}^\phi(\cdot \mid h_n, N_{u_j}) = K_{n+1}^{\phi'}(\cdot \mid h_n, N_{u_j}). \quad (25)$$

The initial conditional distributions at u_j are identical because only Φ is intervened on and the later kernels do not read it. Suppose the laws of H_n agree. For any measurable set D of histories through $n+1$,

$$P^\phi(H_{n+1} \in D \mid N_{u_j}) = \int \mathbf{1}\{(h_n, e) \in D\} K_{n+1}^\phi(de \mid h_n) P^\phi(dh_n) \quad (26)$$

$$= \int \mathbf{1}\{(h_n, e) \in D\} K_{n+1}^{\phi'}(de \mid h_n) P^{\phi'}(dh_n) \quad (27)$$

$$= P^{\phi'}(H_{n+1} \in D \mid N_{u_j}). \quad (28)$$

Induction reaches the attempt endpoint and any outcome measurable there. Integrating over N_{u_j} gives the unconditional version if its distribution is shared.

Propositions 4.2 and 4.3; Corollary 4.4. Each is a direct application of the conjunction in Definition 3.1. Strict RTT requires W3, a nontrivial policy-state transition, and W7, a fresh-proposal difference after non-policy carriers are fixed. A process with constant Φ cannot satisfy W3; a change only to C , M , or W remains fixed away in W7. Increasing compute changes neither logical fact. These are necessary-condition proofs; they make no claim about utility.

A.3 Finite-history emulation and non-identifiability

Theorem 5.1. Let $\mathcal{H}_n$ be the finite set of observable histories through decision n with positive probability under the adaptive mechanism P_{ad} . Hidden state, including Φ_n , may be stochastic conditional on H_n . Define the fixed behavioral kernel

$$q_n(da \mid h_n) = \int \pi_\phi(da \mid h_n) P_{\text{ad}}(d\phi \mid H_n = h_n). \quad (29)$$

This equals $P_{\text{ad}}(A_n \in da \mid H_n = h_n)$ by the law of total probability. The collection $q = (q_1, \dots, q_N)$ is fixed before emulation; it does not modify an internal parameter in response to the realized episode. On zero-probability histories choose any probability kernel.

At decision 1, q_1 matches the adaptive action conditional. The shared environment then matches the next observation conditional. Suppose the observable joint law through decision n matches. Conditioning on any positive-probability h_n , Equation (29) matches the next action, and the environment matches the next observation. Multiplying these conditionals by the matched history probability establishes the induction step. Thus all observable finite-dimensional distributions match. Corollary 5.2 follows because one compatible mechanism contains mutable policy state and another does not.

For a countable but almost surely finite stopping horizon, the construction applies to every finite prefix consistently. We state the finite version in the main text to avoid imposing additional extension conditions that are irrelevant to finite logged episodes.

A.4 Randomized factorial identification

Theorem 5.3. Let $Z_{rf} = \mathbf{1}\{R = r, F = f\}$ and $p_{rf} = \mathbb{P}(Z_{rf} = 1) > 0$. A Horvitz–Thompson version of the cell mean is

$$\tilde{\mu}_{rf} = \frac{1}{N} \sum_{i=1}^N \frac{Z_{i,rf} Q_i^{\text{obs}}}{p_{rf}}. \quad (30)$$

By consistency, $Z_{i,rf}Q_i^{\text{obs}} = Z_{i,rf}Q_i(r, f)$. Randomization makes $Z_{i,rf}$ independent of $Q_i(r, f)$, hence

$$\mathbb{E} \left[\frac{Z_{i,rf}Q_i^{\text{obs}}}{p_{rf}} \right] = \frac{\mathbb{E}Z_{i,rf} \mathbb{E}Q_i(r, f)}{p_{rf}} = \mathbb{E}Q_i(r, f). \quad (31)$$

Averaging proves unbiasedness. Under fixed balanced cell counts, the ordinary within-cell mean has the same conditional expectation. Linearity proves unbiasedness of both contrasts in Equations (10)–(12). If outcomes are missing after assignment, the result applies to intention-to-treat only when missingness is included in the outcome contract or handled under additional stated assumptions.

A.5 Reset isolation

Theorem 5.5 and Corollary 5.6. Let the two arms have common complete state $S_s = s_0$. By the shared-kernel premise, their first post-reset event laws are both $K_{s+1}(\cdot | s_0)$. Assume their history distributions match through event n . Integrating the same K_{n+1} against the same history distribution, as in Equation (24), matches event $n+1$. Induction establishes equality for every post-reset observable. A detected systematic difference contradicts the conjunction of complete-state equality, shared kernels, and no interference; because the erased update is absent from both restored states, it cannot be the isolated cause under those premises.

A.6 Verifier results

Proposition 5.7. For binary variables, $|V - S|$ is exactly the disagreement indicator. Jensen’s inequality for absolute value gives

$$|\mathbb{E}V - \mathbb{E}S| = |\mathbb{E}(V - S)| \leq \mathbb{E}|V - S| = \mathbb{P}(V \neq S). \quad (32)$$

For arms a, b , expand the difference between proxy and truth effects and apply the triangle inequality:

$$|\mathbb{E}(V_a - S_a) - \mathbb{E}(V_b - S_b)| \quad (33)$$

$$\leq |\mathbb{E}(V_a - S_a)| + |\mathbb{E}(V_b - S_b)| \leq \varepsilon_a + \varepsilon_b. \quad (34)$$

Theorem 5.8. Let P_{obs} be the complete observed law. Extend it to model P^+ by defining latent $S := V$ deterministically. Extend the same P_{obs} to P^- by defining $S := 1 - V$. Both extensions integrate back to the identical observed law. For any two arms,

$$\Delta_S^+ = \mathbb{E}[V | a] - \mathbb{E}[V | b], \quad (35)$$

$$\Delta_S^- = \mathbb{E}[1 - V | a] - \mathbb{E}[1 - V | b] = -\Delta_S^+. \quad (36)$$

Thus the observed law admits opposing truth-effect signs whenever the proxy effect is nonzero. A calibration, monotonicity, bounded-error, or external-truth assumption would rule out some extensions, but none is assumed in the theorem.

A.7 Optimization identities

Proposition 6.2. Write $J(\phi) = \int R(z)p_\phi(z | \mathcal{G}) \nu(dz)$. The domination/interchange assumption gives

$$\nabla J(\phi) = \int R(z) \nabla p_\phi(z | \mathcal{G}) \nu(dz) \quad (37)$$

$$= \int R(z)p_\phi(z | \mathcal{G}) \nabla \log p_\phi(z | \mathcal{G}) \nu(dz). \quad (38)$$

Moreover,

$$\int b(\mathcal{G})p_\phi(z | \mathcal{G}) \nabla \log p_\phi(z | \mathcal{G}) \nu(dz) = b(\mathcal{G}) \nabla \int p_\phi dv = 0. \quad (39)$$

Subtracting this zero term proves the identity. If the action support depends on ϕ , boundary terms may appear; that case is outside the proposition.

Lemma 6.3. If $R_i = r$ for all i , then $\bar{R} = r$, $s_R = 0$, and $A_i = (r - r)/\varepsilon = 0$. Any update $\sum_i A_i g_i$ or scalar multiple thereof is zero. Nonlinear algorithms that add terms not weighted by A_i are not covered and must audit those terms separately.

A.8 Budget, drift, and non-uniform safety

Proposition 6.4. Induct on j . The base $B_1 \geq 0$ is assumed. If $B_j \geq 0$ and admission requires $0 \leq c_j \leq B_j$, then $B_{j+1} = B_j - c_j \geq 0$. Telescoping componentwise yields

$$\sum_{i=1}^j c_i = B_1 - B_{j+1} \leq B_1. \quad (40)$$

Theorem 7.1. For each i , Pinsker’s inequality gives

$$\text{TV}(\pi_i, \pi_{i-1}) \leq \sqrt{\frac{1}{2} \text{KL}(\pi_i \| \pi_{i-1})} \leq \sqrt{\kappa_i / 2}. \quad (41)$$

The triangle inequality for the total variation metric gives

$$\text{TV}(\pi_K, \pi_0) \leq \sum_{i=1}^K \text{TV}(\pi_i, \pi_{i-1}) \leq \sum_{i=1}^K \sqrt{\kappa_i / 2}. \quad (42)$$

By the variational characterization of total variation, for any $g \in [0, 1]$, $|\mathbb{E}_{\pi_K} g - \mathbb{E}_{\pi_0} g| \leq \text{TV}(\pi_K, \pi_0)$. Total variation is at most one.

Theorem 7.2. Distinct probability measures differ on at least one measurable set A . If $\pi'(A) < \pi(A)$, $g = \mathbf{1}_A$ has lower expectation under π' . If $\pi'(A) > \pi(A)$, then $\pi'(A^c) = 1 - \pi'(A) < 1 - \pi(A) = \pi(A^c)$ and $g = \mathbf{1}_{A^c}$ works. The constructed utility may not be relevant to a given task; the theorem rules out uniform improvement over the unrestricted bounded-utility class only.

B Symbol and State Dictionary

Table 10: **Canonical symbols (DEFINITION).** Symbols are local to this paper unless explicitly shared across the family.

Symbol	Type	Meaning
X, I	$\mathcal{X} \times I$	Immutable problem and episode identifier.
j, t, n	$\mathbb{N}$	Attempt, within-attempt microstep, and global event indices.
Z_j	$\mathcal{Z}$	Complete marked trajectory for attempt j .
$O_{j,t}, A_{j,t}$	$\mathcal{O}, \mathcal{A}$	Observation available before the next action, and action.
Y_j	$\mathcal{Y}$	Feedback generated from attempt j , with provenance/time.
C_j	$\mathcal{C}$	Prompt and transient context state.
M_j	$\mathcal{M}$	External/resident memory state.
W_j	$\mathcal{W}$	Search tree, files, scratch state, tool/cache workspace.
Φ_j	$\mathcal{P}$	Declared functional policy state mounted for attempt j .
Ξ_j	auxiliary space	Optimizer moments, RNG, routes, schedules, other hidden state.
B_j	$\mathcal{B} \subseteq \mathbb{R}_+^d$	Remaining multidimensional resource budget.
S_j	product space	Complete pre-attempt state in Equation (1).
π_ϕ	stochastic kernel	Action/proposal law parameterized by policy state ϕ .
$U_j, \mathcal{U}_\Theta$	measurable maps	Concrete update and general outer-parameterized update rule.
ρ_j	receipt space	Update inputs/outputs, hashes, times, costs, validation and commit.
$\mathcal{F}_n, \mathcal{F}_{j,t}$	σ -fields	Causally available information at event and microstep.
σ_j, ϵ_j	stopping times	Attempt j start and end.
v_j, u_j	stopping times	Feedback availability and update completion.
τ	stopping time	Episode resolution, abandonment, budget, or safety endpoint.
$\mathfrak{B}_j$	product record	Strict-witness bundle in Equation (7).
R, F	$\{0, 1\}$	Mounted-state and feedback-semantic randomized factors.
$Q(r, f)$	$[0, 1]$	Potential qualification outcome under factorial arm (r, f) .
θ_{state}	$[-1, 1]$	True-feedback post-state versus pre-state effect.
θ_{int}	$[-2, 2]$	State-by-feedback semantic interaction.
S, V	$\{0, 1\}$	True success and proxy verifier decision.

Symbol	Type	Meaning
TV, KL	divergences	Total variation and Kullback–Leibler divergence.
Θ	outer parameter	Meta-learned or engineered update-rule parameters.
$\mathcal{J}(\Theta)$	$\mathbb{R}$	Complete episode-level quality–cost–risk objective.

B.1 Reserved terminology

Attempt. A bounded, version-receipted policy invocation intended to advance or solve the current problem; internal token steps are not automatically separate attempts.

Feedback. A causally available record derived from an attempt or environment interaction. Its presence does not imply truth.

Policy state. A declared, intervenable carrier that directly parameterizes the fresh action/proposal kernel when non-policy carriers are fixed.

Memory state. A typed store read through a memory/retrieval path; mutation alone is RTMA, not strict RTT.

Workspace. Candidate-specific search and tool state, including trees, caches, files, and scratch computations.

Resolution. The registered terminal event for success, abandonment, budget exhaustion, or safety stop.

Strict RTT. A complete W1–W8 functional update–reuse witness, without an implied benefit sign.

RTT benefit. A prespecified causal outcome contrast; separate from mechanism identity.

C Printable Leakage, Circularity, and Claim Audits

C.1 Carrier/reset matrix

Table 11: **Hidden-state and reset matrix (DESIGN PRESCRIPTION).** Each future experiment must replace “declare” with a concrete receipt.

Carrier	Hash/version	Restore or randomize	Failure interpretation
Base weights and adapters	Content hash, mount route, dtype	Swap only registered policy factor	Wrong or bundled intervention
Optimizer/learned-update state	Moments, step, scaler, recurrent code	Restore with candidate/pre-state	Hidden policy carrier
Normalization/routing/gates	Running stats and decisions	Include in policy version or fix	Dead/bypassed update
Prompt/context/KV state	Serialized prompt and cache receipt	Restore before fresh proposal	Context confounding
External/resident memory	Rows, versions, index, retrieval seed	Restore or independent memory factor	Memory confounding
Search/workspace/files	Tree, node scores, files, cache keys	Restore before proposal swap	Search/workspace confounding
Tools and services	Session IDs, remote cache/version	Reset or block-randomize	Environment interference
Randomness and batching	All RNG states, batch peers, scheduler	Common random numbers or assignment	Stochastic imbalance
Verifier/reference	Model/data/config hash, provenance	Freeze before target outcomes	Reward circularity
Budget and stop logic	Ledger hash, thresholds, timers	Equal rules across arms	Differential exposure

C.2 Future-leakage checklist

- L1.** Is every update input timestamped no later than u_j ?
- L2.** Was any later success, hidden test, future tool return, or final answer used to choose an update, checkpoint, learning rate, or gate?
- L3.** Were failed candidates omitted after their later outcomes were known?
- L4.** Does a post- u_j random canary predict the committed delta above the registered tolerance?

- L5.** Were outer-training realized futures physically and logically separated from deployment artifacts?
- L6.** Did asynchronous callbacks complete in the assumed causal order?
- L7.** Are all target-outcome-dependent revisions recorded as a new protocol rather than silently folded into the old one?
- Any “yes” to L2–L4 or L7, or an unresolved answer to L1/L5/L6, blocks a deployed causal-update claim.

C.3 Circularity checklist

- C1.** Is the feedback producer independent of the policy family? If not, is it explicitly labeled a proxy?
- C2.** Is true success externally defined and audited on a disjoint sample?
- C3.** Were reward, state divergence, and final utility reported separately?
- C4.** Were all arms, including stopped and failed updates, included in intention-to-treat reporting?
- C5.** Were hyperparameters and stop thresholds frozen before target-arm inspection?
- C6.** Was mechanism qualification performed irrespective of outcome success?
- C7.** Can the verifier be optimized without task progress (reward hacking), and are adversarial cases included?

C.4 Manuscript claim audit

Claim class	Strongest admitted basis	Explicit exclusion
Strict RTT vocabulary	Authorial definitions and typed interfaces	No priority claim
Separation results	Logical consequences of W1–W8	No performance ranking
Identification results	Randomization/intervention assumptions	No guarantee assumptions hold in a system
Optimization results	Differentiability/support/group assumptions	No finite-step improvement
Safety results	Local KL and bounded-utility assumptions	No global or deployment safety
Related-work map	Verified primary-source protocol descriptions	No replication or independent result validation
MMLA boundary	Public V3 and complete technical-report evidence records	No RTT evidence; no repair of closed overwrite gate
Empirical ladder	Planned protocols and stop rules	No gate executed or passed

Table 12: **Claim–evidence audit.** The strongest paper-level claim is a proved conditional statement or a source-verified protocol description; no empirical RTT success is admitted.